AUTOENCODERS
and
VARIATIONAL AUTOENCODERS

From First Principles to Practice — Complete Edition

Encoders · Decoders · KL Divergence · ELBO · Reparameterisation · VAE Family

Every formula explained symbol by symbol, in simple English, with worked numbers and diagrams

Prepared for: Sunkeerth

Engineering — Robotics, AI and Autonomous Systems

Table of Contents

How to Use This Book 6

The coloured boxes . . . 6

Roadmap of the book . . . 7

Mathematical Prerequisites 8

From calculus . . . 8

From linear algebra . . . 8

From probability . . . 8

PART I 9

Chapter 1: What Problem Are We Actually Solving? 10

1.1 Why compression is the heart of generation . . . 10

1.2 Where the autoencoder fits . . . 11

Chapter 2: The Encoder and the Decoder 12

2.1 The encoder . . . 12

2.2 The decoder . . . 13

Chapter 3: The Plain Autoencoder 14

3.1 Definition . . . 14

3.2 The loss functions . . . 14

3.3 A worked example you can check by hand . . . 16

3.4 The training loop . . . 17

Chapter 4: Varieties of Autoencoders 19

4.1 Undercomplete versus overcomplete . . . 19

4.2 Denoising autoencoder (DAE) . . . 19

4.3 Sparse autoencoder . . . 20

4.4 Contractive autoencoder (CAE) . . . 21

4.5 Summary . . . 21

Chapter 5: Why Plain Autoencoders Cannot Generate 22

5.1 The tempting idea . . . 22

5.2 The real reason: an unstructured latent space . . . 22

5.3 The two properties we need . . . 23

5.4 The big idea behind VAEs . . . 23

PART II 24

Chapter 6: Random Variables and Gaussian Distributions **25**

6.1 What a distribution really represents . . . 25

6.2 Sampling . . . 26

Chapter 7: KL Divergence **27**

7.1 The definition . . . 27

7.2 Why asymmetry matters . . . 28

7.3 The one formula you actually need . . . 28

Chapter 8: Bayes' Rule and Latent Variable Models **31**

8.1 Bayes' rule . . . 31

8.2 Latent variable models . . . 32

8.3 The problem: p(x) is intractable . . . 32

PART III **34**

Chapter 9: Setting Up the Variational Problem **35**

9.1 Restating the goal . . . 35

9.2 Introducing an approximate posterior . . . 35

Chapter 10: Deriving the ELBO, Step by Step **37**

10.1 Step 1: insert the encoder without changing anything . . . 37

10.2 Step 2: use Bayes' rule, then add and subtract the same thing . . . 37

10.3 Step 3: recognise the two pieces . . . 37

10.4 Step 4: why this is exactly what we need . . . 38

10.5 Step 5: split the ELBO into two practical terms . . . 39

10.6 From ELBO to the loss you code . . . 40

Chapter 11: The Reparameterisation Trick **41**

11.1 The problem: sampling is not differentiable . . . 41

11.2 The fix: move the randomness outside . . . 41

11.3 Why not use another gradient method? . . . 42

Chapter 12: Full VAE Architecture and Implementation **44**

12.1 The complete picture . . . 44

12.2 Why the encoder outputs log σ^2 . . . 44

12.3 Full PyTorch implementation . . . 45

12.4 Mapping every formula to a line of code . . . 46

12.5 Generating new samples after training . . . 47

Chapter 13: Training Dynamics and Common Failure Modes **48**

13.1 What healthy training looks like . . . 48

13.2 The reconstruction versus regularisation tug-of-war . . . 48

13.3 Posterior collapse . . . 49
13.4 Common fixes . . . 49

Chapter 14: A Complete Worked Numerical Example 51

14.1 Setup . . . 51
14.2 Recover the widths . . . 51
14.3 Sample z with the reparameterisation trick . . . 51
14.4 Reconstruction loss . . . 52
14.5 KL term . . . 52
14.6 The final loss . . . 53

Chapter 15: Convolutional VAEs for Real Image Data 54

15.1 Why flatten-and-linear does not scale . . . 54
15.2 Code sketch . . . 54
15.3 Practical tips . . . 56

PART IV 57

Chapter 16: Conditional VAE (CVAE) 58

16.1 The limitation of a plain VAE . . . 58
16.2 The fix: condition everything on a label . . . 58

Chapter 17: β-VAE and Disentanglement 60

17.1 What disentanglement means . . . 60
17.2 The β-VAE loss . . . 60
17.3 The trade-off . . . 61

Chapter 18: VQ-VAE — Discrete Latent Codes 62

18.1 Why discrete latents? . . . 62
18.2 How it works . . . 62
18.3 Codebook collapse . . . 63

Chapter 19: VAEs vs. GANs vs. Diffusion vs. Flows 64

19.1 GANs . . . 64
19.2 Diffusion models . . . 64
19.3 Normalising flows . . . 64
19.4 Comparison . . . 65
19.5 The deep connection: latent diffusion . . . 65

PART V 66

Chapter 20: VAEs in Robotics, RL, and Autonomous Systems 67

20.1 World models . . . 67

20.2 Latent action spaces . . . 68
20.3 Anomaly detection and the sim-to-real gap . . . 68

Chapter 21: Latent Space Interpolation and Visualisation **69**

Chapter 22: Where to Go From Here **70**

PART VI **71**

Chapter 23: Tighter Bounds — IWAE and Hierarchical VAEs **72**

23.1 The looseness of the plain ELBO . . . 72
23.2 Importance-Weighted Autoencoders (IWAE) . . . 72
23.3 Hierarchical VAEs . . . 73

Chapter 24: Evaluating Generative Models **74**

24.1 Held-out reconstruction and ELBO . . . 74
24.2 Fréchet Inception Distance (FID) . . . 74
24.3 Always look . . . 75

Chapter 25: Exercises and Practice Problems **76**

25.1 Conceptual questions . . . 76
25.2 Computation practice . . . 76
25.3 Design questions . . . 76
25.4 Solutions . . . 77

APPENDICES **78**

Appendix A: Frequently Asked Questions **79**

Appendix B: Notation Reference **80**

Appendix C: Glossary **81**

Appendix D: Full Formula Cheat Sheet **82**

Appendix E: Debugging Checklist for VAE Code **83**

Appendix F: Further Reading **84**

Appendix G: The Whole Book in One Page **85**

How to Use This Book

Read this page first: it explains the coloured boxes you will see everywhere.

This book answers one question completely: how do we go from a plain Autoencoder to a Variational Autoencoder (VAE), and why does that jump matter? You already know neural networks, gradients and training loops from your Transformer work. What is new here is a way of thinking about a network as a *probability machine* instead of just a function.

Every idea is built in the same order: first a plain-English explanation, then a simple analogy, then a diagram, then the formula with every symbol explained, then a worked example with real numbers, and finally code. If you ever get lost in a formula, go back to the plain-English sentence just before it. The formula never says anything more than that sentence; it only says it more precisely.

The coloured boxes

The boxes are designed so that you can revise the whole book by reading only the boxes. Yellow highlighted text inside normal paragraphs, like this sentence, marks the single most important statement in that paragraph.

KEY IDEA
The one idea from a section that you must not forget. If you only read these boxes you would still understand the book.

IMPORTANT - FOCUS HERE
Something that is easy to get wrong in exams, interviews or code. Slow down when you see red.

FORMULA — Blue boxes hold formulas

formula written clearly

What each symbol means

symbol	what it means, in simple English

In plain words: every formula box ends with a sentence that reads the formula aloud.

WORKED EXAMPLE
Real numbers pushed through the formula, step by step, so you can check every line with a calculator.

SIMPLE ANALOGY
An everyday comparison that gives you a mental picture before the maths.

WHY DOES THIS WORK?
The reason behind a design choice. Understanding the *why* is what separates top students from the rest.

ROBOTICS CONNECTION
Where the idea shows up in robotics, simulation and autonomous systems, which is your area.

Roadmap of the book

Part	What you learn	Chapters
I. Foundations	Encoder, decoder, autoencoder, its variants, and why it cannot generate	1 – 5
II. Probability toolkit	Gaussians, sampling, KL divergence, Bayes' rule, latent variable models	6 – 8
III. The VAE	Variational inference, ELBO derivation, reparameterisation trick, full code, training, worked example	9 – 15
IV. VAE family	Conditional VAE, β-VAE, VQ-VAE, and comparison with GANs, diffusion and flows	16 – 19
V. Applications	World models, latent actions, anomaly detection, interpolation, next steps	20 – 22
VI. Advanced	IWAE, hierarchical VAEs, evaluation metrics, exercises with solutions	23 – 25
Appendices	FAQ, notation, glossary, formula cheat sheet, debugging checklist, reading list	A – G

NOTE ON THIS EDITION
This edition expands the original notes and fixes a few problems in them:

- The Greek letters are now used consistently everywhere: φ (phi) always belongs to the ENCODER and θ (theta) always belongs to the DECODER. The first version used θ for the encoder in Chapter 2 and then switched. The convention here matches the original Kingma and Welling paper.
- Wrong chapter cross-references (for example, β-VAE being called 'Chapter 14') are corrected.
- A new chapter on the encoder and decoder on their own (Chapter 2) and a new hand-computable autoencoder example have been added.
- Every formula now has a symbol table and a plain-English reading. Many new diagrams, extra worked examples and more exercises have been added.

Mathematical Prerequisites

Only these tools are used. Each is explained again the first time it appears.

From calculus

The chain rule tells you how a change travels through a chain of functions. If z depends on μ, and the loss depends on z, then the change in loss per change in μ equals (change of loss per change of z) multiplied by (change of z per change of μ). Backpropagation is nothing more than the chain rule applied over and over, from the loss back to every weight. You need this to understand why the reparameterisation trick in Chapter 11 is necessary.

Partial derivatives measure how the output changes when you nudge only one input and hold all others fixed. Every gradient in this book is a partial derivative with respect to one weight.

From linear algebra

Inputs x, latent codes z, and the encoder outputs μ and σ are all **vectors**, which are simply lists of numbers. A **dot product** multiplies two lists element by element and adds the results. Every fully connected layer is a matrix-vector multiplication (many dot products at once) followed by a non-linearity such as ReLU.

From probability

FORMULA — Expectation

$$E_{z\sim q}[\, f(z)\,] \approx (1/N) \cdot \Sigma_i\, f(z_i), \text{ with } z_i \text{ drawn from } q$$

What each symbol means

E[·]	expectation: a weighted average
z~q	z is drawn at random according to the distribution q
f(z)	any quantity that depends on z, for example a loss

In plain words: the expectation is the average value of f(z) you would get if you drew z from q many, many times. In practice we estimate it by drawing a few samples and averaging, and VAEs often use just one sample.

Linearity of expectation says E[a + b] = E[a] + E[b]. It lets us split one big expectation into two smaller ones, which is exactly how the ELBO is split into a reconstruction part and a KL part in Chapter 10.

PART I

Foundations: rebuilding the autoencoder from scratch

Chapter 1: What Problem Are We Actually Solving?

Discriminative versus generative models, and the idea of a latent space.

Most models you have used so far, including Transformers used for classification or translation, are **discriminative**. A discriminative model receives an input and predicts something about it: a label, a translation, or the next token. It learns the *boundary* between classes, but it has no idea what a realistic input looks like as a whole.

This book is about a different family called **generative models**. A generative model learns what the data itself looks like, so well that it can create brand-new, realistic examples that were never in the training set. Formally, it tries to learn the probability distribution p(x) from which the training data was drawn.

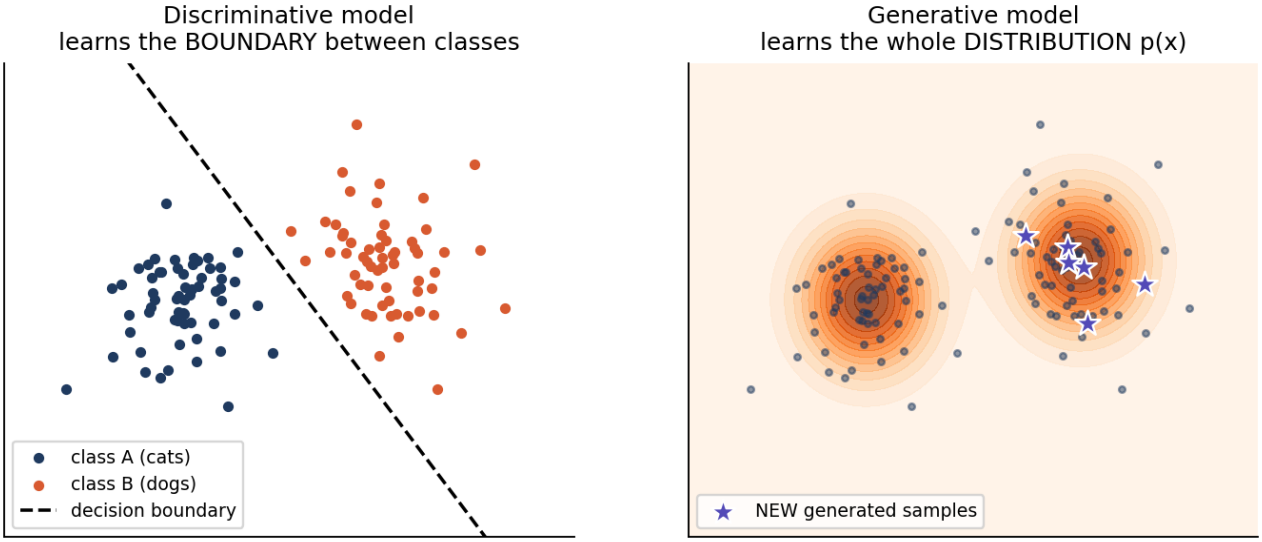


Figure 1. *Left: a discriminative model only learns a line that separates classes. Right: a generative model learns where data is dense (orange), so it can draw new points (stars) from those dense regions.*

1.1 Why compression is the heart of generation

To generate something new, a model first needs a compact and organised way to describe what it has seen. Imagine describing every photograph of a face with only 20 numbers instead of 150,000 pixel values. One number might roughly control the smile, another the age, another the lighting. If such a description exists and is well organised, you can simply choose new values for those 20 numbers and turn them back into a face that never existed.

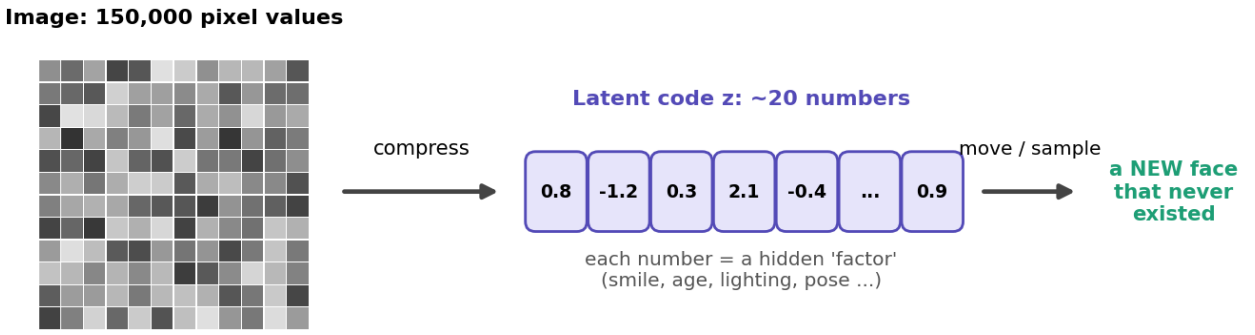


Figure 2. *The central idea of the whole book: compress data into a few meaningful numbers (the latent code), then move around in that small space to create new data.*

The small list of numbers is called the **latent representation** or **latent code**, usually written z. The space where all possible codes live is called the **latent space**. The word *latent* means hidden: nobody labels these factors for the network; it discovers them by itself.

KEY IDEA
A generative model does not memorise the training data. It learns the underlying distribution the data came from, so that it can draw *new* samples from the same distribution. Everything in this book is about two things: learning a good latent space, and learning how to sample from it safely.

SIMPLE ANALOGY
Think of a music producer who has listened to thousands of songs. A discriminative producer can only tell you 'this is jazz' or 'this is rock'. A generative producer understands the rules of jazz well enough to compose a new jazz song. The latent space is like the producer's mental control panel: tempo, mood, instruments, key.

1.2 Where the autoencoder fits

The autoencoder is the simplest network that learns a compact representation. On its own it is *not* a good generator (Chapter 5 explains exactly why), but it introduces all three building blocks the VAE needs: an encoder, a bottleneck, and a decoder. Once these are clear, the VAE will feel like a small, well-motivated upgrade instead of a wall of new notation.

Chapter 2: The Encoder and the Decoder

Two networks with opposite jobs: one compresses, one rebuilds.

Before joining them into an autoencoder, it helps to understand each half separately. An encoder turns something big into something small, and a decoder turns something small back into something big. Everything else is detail.

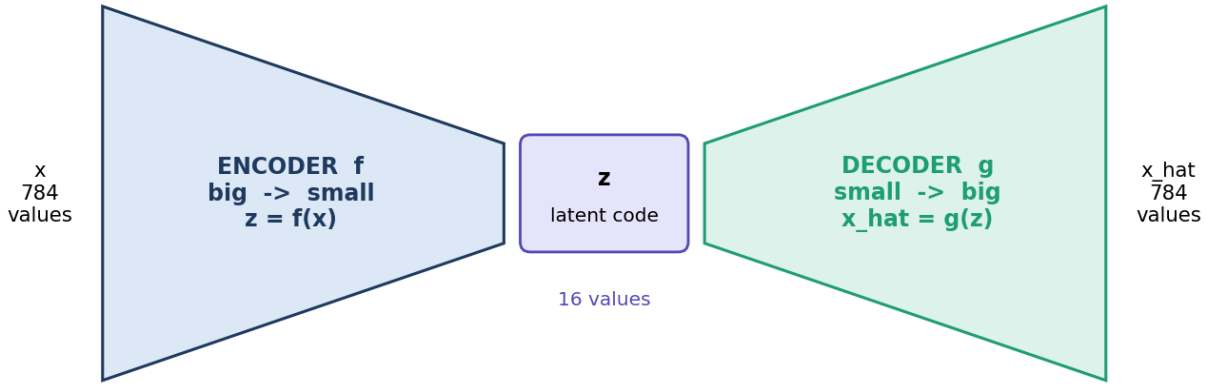


Figure 3. *The encoder narrows 784 numbers down to 16; the decoder widens 16 numbers back to 784.*

2.1 The encoder

An encoder is a neural network that takes a large input, for example a 28 by 28 MNIST image with 784 pixel values, and outputs a short vector, for example 16 numbers. Inside, it is an ordinary stack of layers (fully connected, convolutional, or anything suited to the data), where each layer is a little smaller than the one before it.

FORMULA — The encoder

$$z = f_{\varphi}(x)$$

What each symbol means

x	the input: an image, a sensor frame, a LiDAR scan, a sentence embedding
f	the encoder network (a function made of layers)
φ (phi)	all the learnable weights and biases inside the encoder
z	the latent code: a short vector that summarises x

In plain words: pass the input x through the encoder network, whose behaviour is set by its weights φ, and you get the short summary vector z.

2.2 The decoder

A decoder does the opposite. It takes the short vector z and outputs something the same size as the original input. Its layers usually mirror the encoder: each layer is a little *larger* than the one before it.

FORMULA — The decoder

$$\hat{x} = g_\theta(z)$$

What each symbol means

z	the latent code coming out of the encoder
g	the decoder network
θ (theta)	all the learnable weights inside the decoder
x̂ (x-hat)	the reconstruction. The little hat means 'estimate of x', not x itself

In plain words: pass the code z through the decoder network, whose behaviour is set by θ, and you get x-hat, the decoder's best guess of what the original input looked like.

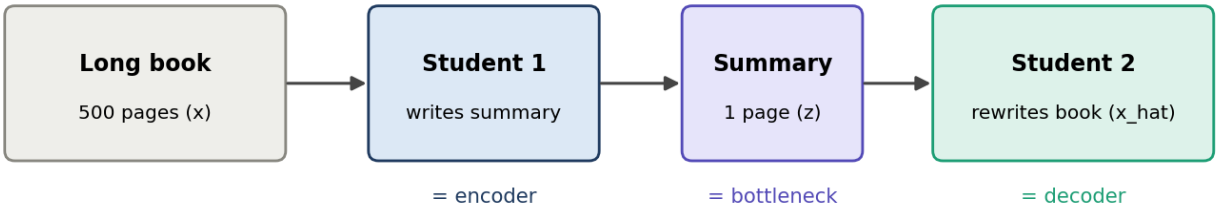


Figure 4. *The book-summary analogy: the summary must be short, so only the important ideas survive.*

SIMPLE ANALOGY

Student 1 reads a 500-page book and writes a one-page summary. Student 2 never sees the book; they only get the summary and must rewrite the whole book from it. If the summary is good, the rewritten book is close to the original. Student 1 is the encoder, the summary is z, and Student 2 is the decoder. Because the summary is limited to one page, Student 1 is *forced* to keep only the ideas that matter most.

REMEMBER THIS

Encoder = compress = x → z. Decoder = rebuild = z → x̂. Encoder weights are φ, decoder weights are θ. A handy memory trick: **φ** sounds like 'find' (the encoder *finds* the code), **θ** is used for the generative model (the decoder *generates*).

Chapter 3: The Plain Autoencoder

Architecture, loss function, a hand-computed example, and working PyTorch code.

3.1 Definition

An **autoencoder** (AE) is an encoder and a decoder joined together and trained as one network. The target output is the input itself. At first this sounds pointless: why not just copy the input? The trick is the **bottleneck**, a middle layer with far fewer numbers than the input. Because not every detail can squeeze through, the network must discover the most important, compressible structure in the data in order to rebuild it well.

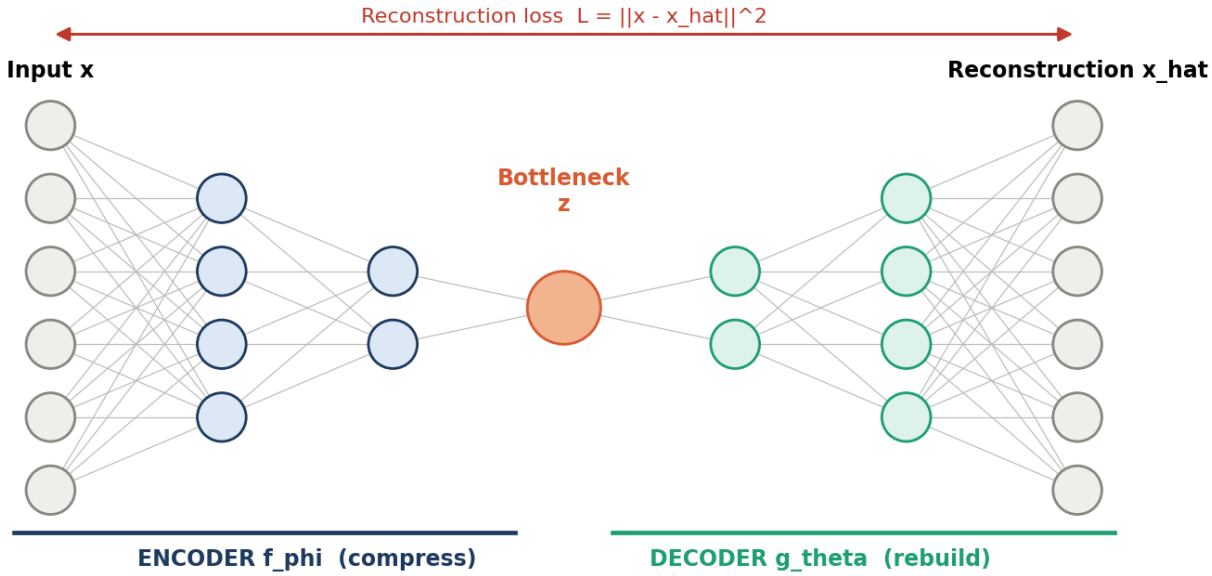


Figure 5. *A plain autoencoder. Wide on both ends, narrow in the middle. The loss compares the two ends.*

Because the input is its own target, no human labels are needed. This is called **self-supervised learning**: we manufacture a supervised-looking problem (predict x from x) out of unlabelled data, purely to force useful structure to appear in the bottleneck.

3.2 The loss functions

Training needs one number that says how wrong the reconstruction is. This number is the **loss**. Two losses are common, and which one you pick depends on the type of data.

FORMULA — Mean Squared Error (MSE)

$$L(x, \hat{x}) = \|x - \hat{x}\|^2 = \Sigma_i \, (x_i - \hat{x}_i)^2$$

What each symbol means

x_i	the true value at position i (for example the brightness of pixel i)
$\hat{x}_i$	the reconstructed value at the same position
$x_i - \hat{x}_i$	the error at that position
$( )^2$	squaring: makes every error positive and punishes big errors much more than small ones
Σ_i	add up over every position i
$\| \cdot \|^2$	squared length of a vector; just a short way of writing the same sum

In plain words: for every pixel, measure how far the guess is from the truth, square it, and add all of these up.

Why it is built this way: squaring means an error of 0.4 costs 16 times more than an error of 0.1, so the network works hardest on its worst mistakes. It is also smooth and easy to differentiate.

FORMULA — Binary Cross-Entropy (BCE)

$$L(x, \hat{x}) = -\Sigma_i \, [\, x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i) \,]$$

What each symbol means

x_i	true value, between 0 and 1
$\hat{x}_i$	predicted value between 0 and 1 (from a sigmoid output)
log	natural logarithm; log of a number near 0 is a large negative number
leading minus	turns the negative logs into a positive loss

In plain words: if the true pixel is 1, only the part $-\log(\hat{x})$ is active, and it becomes huge when the network predicts something close to 0. If the true pixel is 0, only $-\log(1 - \hat{x})$ is active, and it becomes huge when the network predicts close to 1. So BCE punishes *confident wrong* answers very heavily.

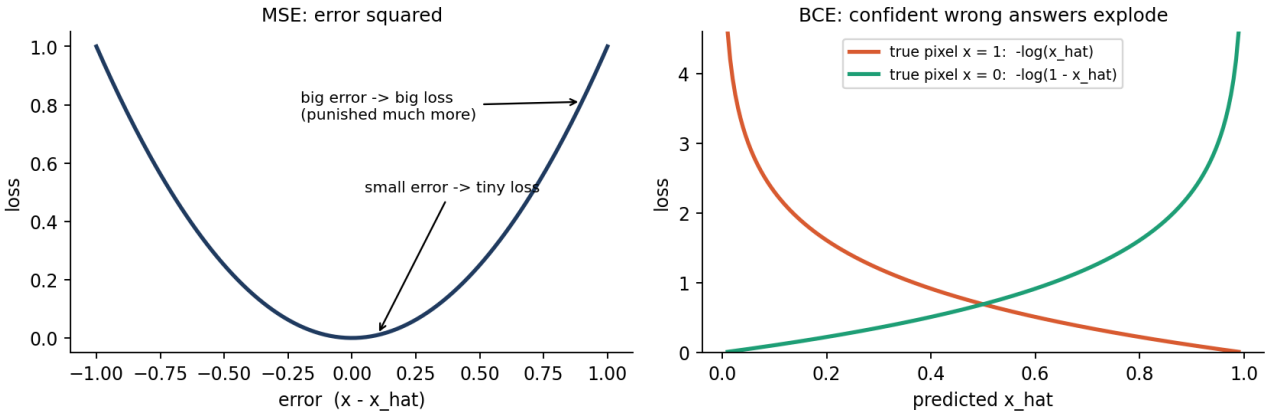


Figure 6. *Left: MSE grows with the square of the error. Right: BCE shoots to infinity when the prediction is confidently on the wrong side.*

IMPORTANT - FOCUS HERE
Match the loss to the output layer. Use a **sigmoid output + BCE** for data in [0, 1] (like MNIST). Use a **linear output + MSE** for unbounded real values (like joint angles or depth). Mixing them, for example BCE on values outside [0, 1], produces NaN losses or nonsense.

3.3 A worked example you can check by hand

Let us make the smallest possible autoencoder: three input numbers squeezed into *one* latent number, and then rebuilt. The weights are chosen by hand so every step is easy to verify.

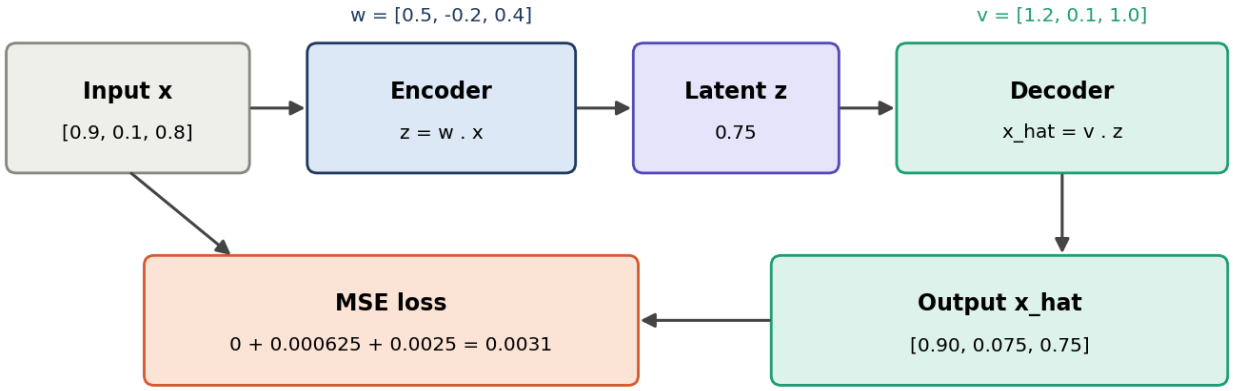


Figure 7. *The full forward pass and loss of a tiny autoencoder with hand-picked weights.*

WORKED EXAMPLE
Input x = [0.9, 0.1, 0.8]. Encoder weights w = [0.5, −0.2, 0.4]. Decoder weights v = [1.2, 0.1, 1.0].

- **Step 1 — encode (dot product):** z = 0.5×0.9 + (−0.2)×0.1 + 0.4×0.8 = 0.45 − 0.02 + 0.32 = **0.75**. Three numbers became one.
- **Step 2 — decode:** $\hat{x}$ = [1.2×0.75, 0.1×0.75, 1.0×0.75] = **[0.90, 0.075, 0.75]**.
- **Step 3 — errors:** 0.9 − 0.90 = 0, 0.1 − 0.075 = 0.025, 0.8 − 0.75 = 0.05.
- **Step 4 — square and add:** $0^2 + 0.025^2 + 0.05^2 = 0 + 0.000625 + 0.0025 =$ **0.003125**.
- **Step 5 — learn:** backpropagation now computes, for each of the 6 weights, how much the loss would change if that weight moved a little, and moves each weight slightly in the direction that lowers the loss.

Notice what happened in Step 2: the decoder can only produce outputs that are *multiples of the same vector v*. With a one-number bottleneck, the network can only describe inputs along one direction. That is the bottleneck at work: it forces the network to choose the single most useful direction in the data.

WHY DOES THIS WORK?
With linear layers and MSE loss, an autoencoder learns the same subspace as Principal Component Analysis (PCA). Adding non-linear activations such as ReLU lets it learn curved, non-linear structure, which is why deep autoencoders can compress images far better than PCA.

3.4 The training loop

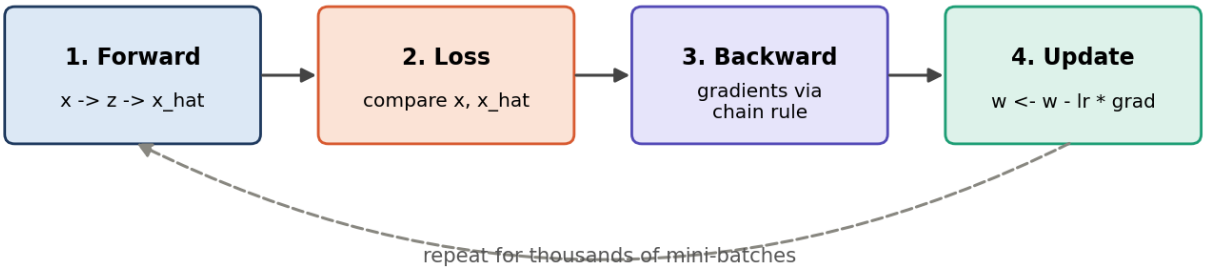


Figure 8. *Every training step does the same four things.*

PyTorch: a complete MLP autoencoder for 28x28 images

```
import torch
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, latent_dim=16):
        super().__init__()
        self.encoder = nn.Sequential(            # f_phi : 784 -> 16
            nn.Flatten(),
            nn.Linear(28*28, 256), nn.ReLU(),
            nn.Linear(256, 64),    nn.ReLU(),
            nn.Linear(64, latent_dim)              # the bottleneck
        )
        self.decoder = nn.Sequential(            # g_theta : 16 -> 784
            nn.Linear(latent_dim, 64), nn.ReLU(),
            nn.Linear(64, 256),        nn.ReLU(),
            nn.Linear(256, 28*28),     nn.Sigmoid(),   # outputs in [0,1]
            nn.Unflatten(1, (1, 28, 28))
        )
    def forward(self, x):
        z = self.encoder(x)
        x_hat = self.decoder(z)
        return x_hat, z

model = Autoencoder(latent_dim=16)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()

for x, _ in dataloader:            # labels are ignored: self-supervised
    x_hat, z = model(x)            # 1. forward
    loss = loss_fn(x_hat, x)       # 2. loss: compare output to INPUT
    optimizer.zero_grad()
    loss.backward()                # 3. backward (chain rule)
    optimizer.step()               # 4. update weights
```

KEY IDEA

Remember two facts about this code, because both change in the VAE: (1) the encoder outputs a **single fixed vector z**, not a distribution; and (2) nothing in the loss says anything about the overall shape of the latent space.

Chapter 4: Varieties of Autoencoders

Ways to stop the network from cheating and to shape the latent code.

The plain autoencoder is the baseline. The variants below each add one extra pressure, either to prevent the network from learning a useless copy of the input, or to give the latent code a desirable property.

4.1 Undercomplete versus overcomplete

An autoencoder is **undercomplete** when the latent size is smaller than the input size. This is the standard setup: the size gap itself forces compression. It is **overcomplete** when the latent size is equal to or larger than the input size. An overcomplete autoencoder with no extra rule can simply learn the identity function and copy input to output, learning nothing useful. The next three variants are ways to make large latent layers useful.

4.2 Denoising autoencoder (DAE)

Instead of giving the encoder the clean input, we first corrupt it (add random noise, or blank out random pixels), and then ask the decoder to reproduce the **clean** original. A corrupted pixel alone does not reveal its true value, so the network must use the surrounding context and the overall structure of the object to fill in the gaps.

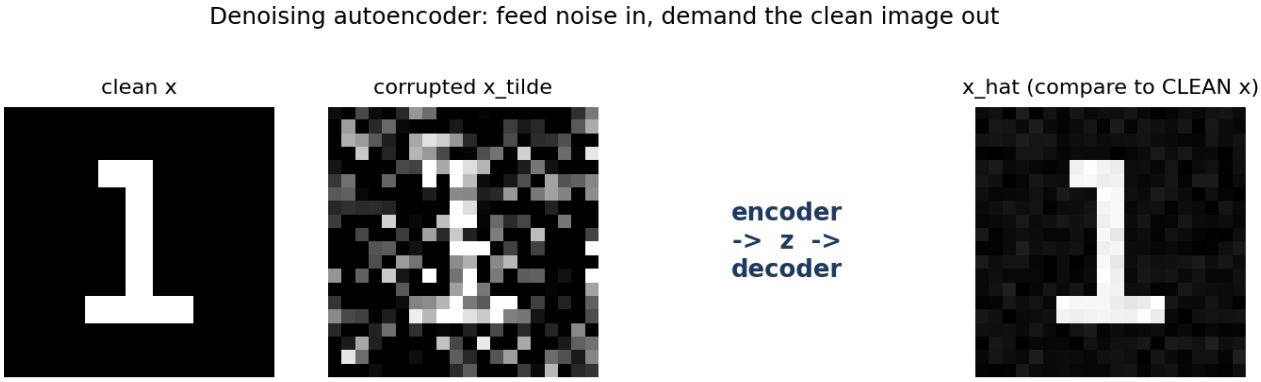


Figure 9. *A denoising autoencoder: noisy input in, clean image required out.*

FORMULA — Denoising autoencoder

$$\tilde{x} = \text{corrupt}(x),\ \hat{x} = g_\theta(f_\varphi(\tilde{x})),\ L = \|x - \hat{x}\|^2$$

What each symbol means

$\tilde{x}$ **(x-tilde)**	the corrupted version of x
x	the clean original, used as the target

In plain words: feed in the damaged version, but compare the output with the undamaged version.

COMMON MISTAKE
Comparing the output with the *corrupted* input $\tilde{x}$ instead of the clean x. That would teach the network to reproduce noise, which defeats the whole purpose.

ROBOTICS CONNECTION
Denoising autoencoders are used to clean depth-camera and LiDAR data, which often has holes and speckle noise, before passing it to a planner.

4.3 Sparse autoencoder

Here the latent layer may be large, but we add a penalty that pushes most latent units towards zero for any given input, so only a few units are active at once. Each unit then tends to specialise in one kind of feature, which makes features easier to interpret.

FORMULA — Sparsity penalty (L1 version)

$$L = \|x - \hat{x}\|^2 + \lambda \cdot \Sigma_j |z_j|$$

What each symbol means

z_j	activation of latent unit j
$\|z_j\|$	absolute value, so both positive and negative activations cost
λ (lambda)	how strongly sparsity is enforced; a hyperparameter you tune

In plain words: rebuild well, but pay a small price for every unit you switch on.

A second common version uses a KL divergence penalty (Chapter 7) that pushes the average activation of each unit towards a small target ρ such as 0.05, meaning each unit should be active on roughly 5% of inputs.

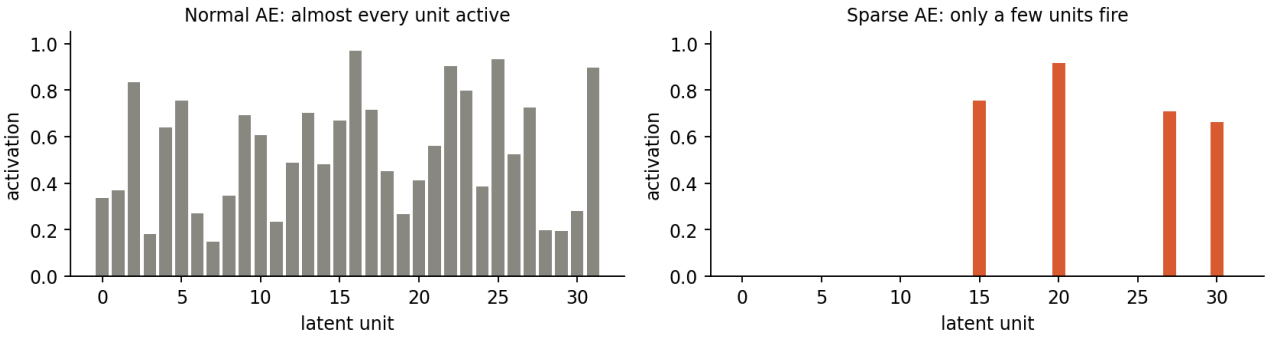


Figure 10. *Dense code versus sparse code: in a sparse code only a handful of units fire for a given input.*

4.4 Contractive autoencoder (CAE)

A contractive autoencoder adds a penalty on how sensitive the code is to tiny changes in the input. The sensitivity is measured by the Jacobian matrix J, whose entries are $\partial z_j/\partial x_i$: how much latent unit j moves when input i is nudged.

FORMULA — Contractive penalty

$$L = \|x - \hat{x}\|^2 + \lambda \cdot \|J_f(x)\|_F^2$$

What each symbol means

$J_f(x)$	Jacobian of the encoder: all the partial derivatives of z with respect to x
$\|\cdot\|_F^2$	Frobenius norm squared: the sum of squares of every entry of the matrix

In plain words: rebuild well, but keep the code locally flat, so small meaningless wiggles in the input barely move z.

4.5 Summary

Variant	What changes	Why it helps
Undercomplete AE	Latent size < input size	Compression is forced by the size gap
Overcomplete AE	Latent size ≥ input size	Needs an extra rule, otherwise it just copies
Denoising AE	Input is corrupted, target stays clean	Learns robust, context-aware features
Sparse AE	Penalty on how many units fire	Specialised, interpretable units
Contractive AE	Penalty on the encoder's Jacobian	Code is stable under small input changes

KEY IDEA

None of these variants make the autoencoder a good generator. They improve the *features*, but none of them says what the latent space should look like *as a whole*. That is the missing piece, explained next.

Chapter 5: Why Plain Autoencoders Cannot Generate

The hinge of the whole book. Understand this and the VAE becomes obvious.

5.1 The tempting idea

After training, the decoder turns latent codes into realistic outputs, as long as those codes came from encoding real inputs. So here is the tempting idea: pick a *random* vector z, feed it to the decoder, and get a brand-new sample. In practice this fails badly. The outputs look like noise, blurry blends of unrelated objects, or plain garbage.

5.2 The real reason: an unstructured latent space

Look at what the loss actually asks for. The only requirement is that each individual training point maps to some code from which the decoder can rebuild it. The loss says nothing about the empty space between those codes. The encoder is therefore free to place all the zeros in one tight cluster, all the ones in another cluster far away, and leave big empty regions in between. The decoder never receives any code from those empty regions during training, so it has never learned what to output there.

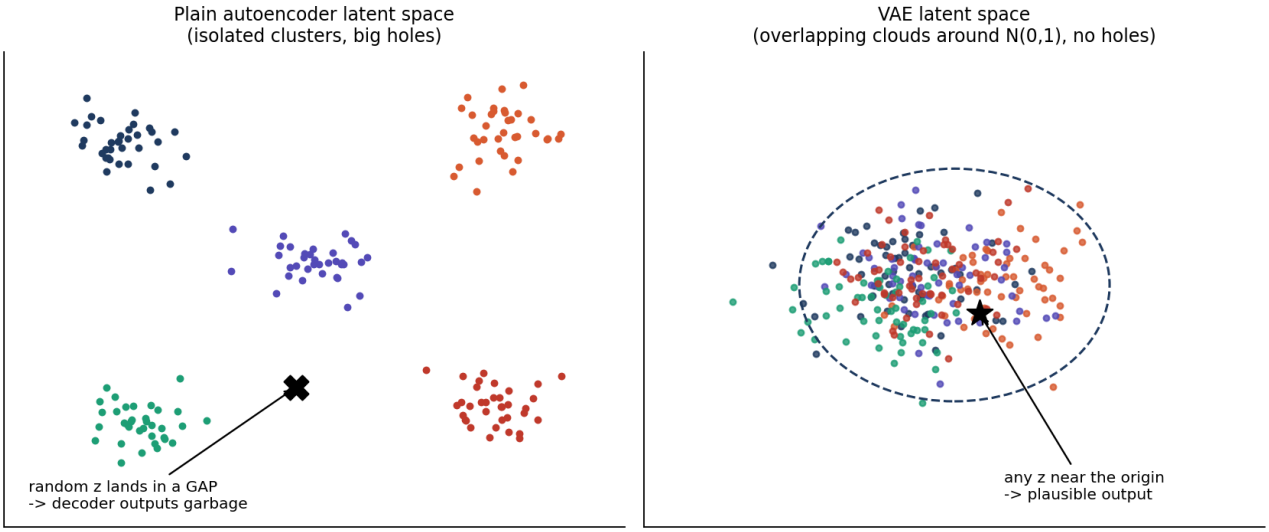


Figure 11. *Left: a plain autoencoder's latent space has isolated clusters and holes, so a random code (X) decodes to garbage. Right: a VAE's latent space is filled and centred, so any point near the origin (star) decodes to something plausible.*

This is not a bug of one particular network. It is a structural consequence of the loss function. We never asked the latent space to be smooth, continuous or centred anywhere, so it is not.

SIMPLE ANALOGY

Imagine a city map where houses are only built in five separate villages with empty desert between them. If someone drops you at a random GPS point, you will probably land in the desert, where there is nothing. A VAE is like a city planning law that says every village must spread out and grow toward the city centre until the villages touch. Then almost any random point lands in a real neighbourhood.

5.3 The two properties we need

IMPORTANT - FOCUS HERE

For random sampling to work, the latent space must have:

- **Continuity** — codes that are close together must decode to outputs that look similar. No sudden cliffs into garbage.
- **Completeness** — any code drawn from a simple, known distribution (such as a standard Gaussian) must decode to something sensible, not only the exact codes seen in training.

5.4 The big idea behind VAEs

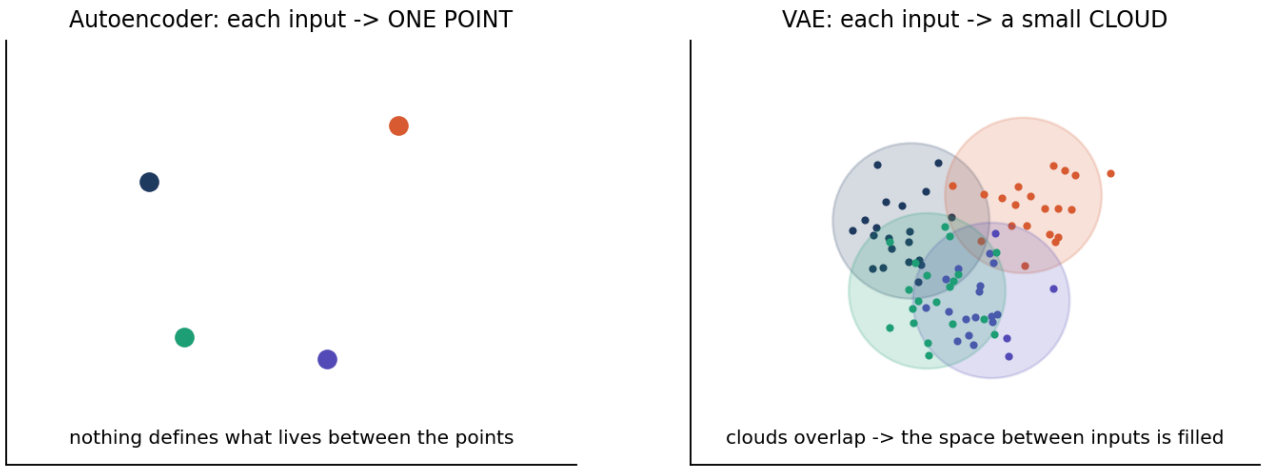


Figure 12. *The single conceptual change from autoencoder to VAE: points become clouds, and the clouds overlap.*

KEY IDEA

Instead of mapping each input to one fixed point, map each input to a small cloud (a probability distribution) of nearby points, and push all these clouds to overlap one shared reference distribution, the standard Gaussian N(0, 1). Overlapping clouds fill the gaps, which gives continuity. Centring them on N(0, 1) tells us exactly where to sample from, which gives completeness.

That single idea is the entire conceptual leap from autoencoder to VAE. Everything in Part III is the mathematics needed to make it precise and trainable. First, Part II builds the small amount of probability theory needed to follow it.

PART II

The probability toolkit: only the four ideas you actually need

Chapter 6: Random Variables and Gaussian Distributions

What a distribution is, what μ and σ do, and what sampling means.

6.1 What a distribution really represents

A **random variable** is a quantity whose value is uncertain, like the result of a dice roll. A **probability distribution** describes how likely each possible value is. For continuous values we use a **density**: a curve whose height at a point says how relatively likely values near that point are. The total area under a density curve is always exactly 1, because *something* must happen.

We use the **Gaussian** (or **Normal**) distribution almost everywhere, because it is simple, well understood, and has formulas that make training easy.

FORMULA — The 1-D Gaussian density

$$N(z\,;\,\mu,\sigma^2) = 1/(\sigma\sqrt{(2\pi)}) \cdot \exp(\,-(z-\mu)^2 / (2\sigma^2)\,)$$

What each symbol means

μ (mu)	the mean: the centre of the bell curve
σ² (sigma squared)	the variance: how spread out the bell is
σ	the standard deviation: the square root of the variance, in the same units as z
exp(−(z−μ)²/(2σ²))	gets smaller the further z is from μ, which makes the bell shape
1/(σ√(2π))	a scaling factor so the total area is exactly 1

In plain words: values near the centre μ are most likely, and the likelihood falls off smoothly the further you go. A small σ gives a tall, narrow bell; a large σ gives a short, wide bell.

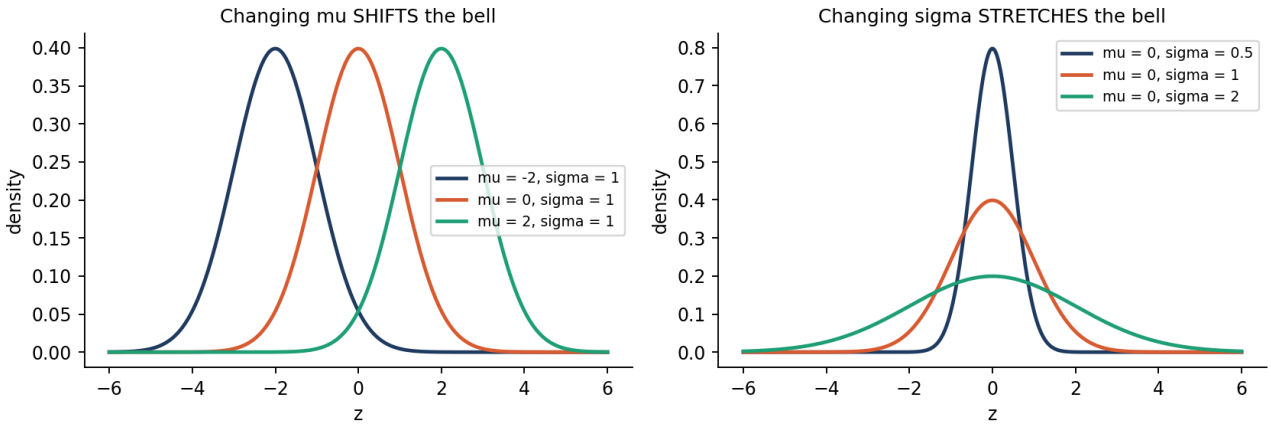


Figure 13. *Left: μ slides the bell left and right. Right: σ makes it narrower (taller) or wider (flatter).*

The **standard normal** N(0, 1) is the special case with μ = 0 and σ = 1. It is the reference shape used as the VAE prior. A **multivariate** Gaussian with a diagonal covariance simply

repeats this independently for each dimension: a 16-dimensional latent code has 16 centres and 16 widths.

KEY IDEA
In a VAE, every input x gets its own little Gaussian bump in latent space, with its own μ and its own σ, both produced by the encoder. The prior N(0, 1) is the one big bump that all of these little bumps are pulled towards.

6.2 Sampling

Sampling means drawing one concrete value out of a distribution. Writing z ~ N(0, 1) means 'z is drawn from the standard normal'. Most draws land near 0; draws far away are rare. Running the draw twice gives two different numbers. The more samples you draw, the more their histogram looks like the density curve.

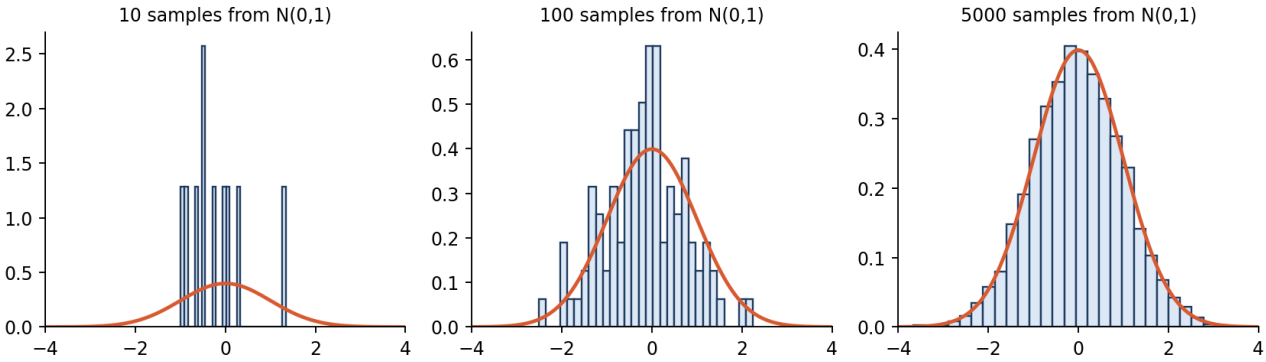


Figure 14. *With few samples the histogram is rough; with many samples it matches the bell curve (orange).*

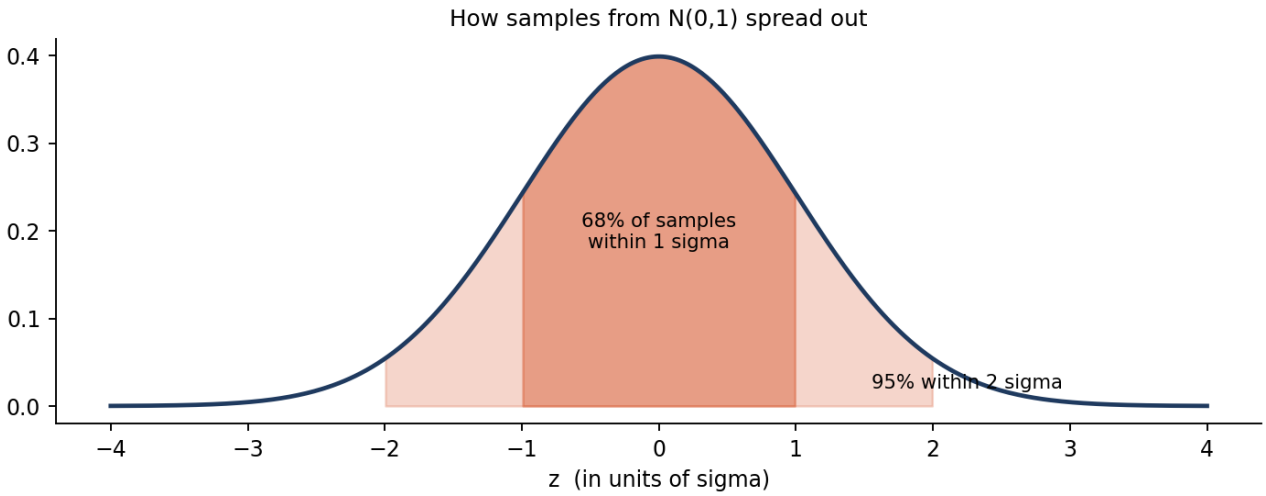


Figure 15. *About 68% of standard-normal samples fall within ±1 and about 95% within ±2. So sampling z ~ N(0,1) almost always gives values between −2 and 2.*

WHY DOES THIS WORK?
Sampling is random, and randomness has no derivative. This creates a real problem for backpropagation, which Chapter 11 solves with the reparameterisation trick. Keep this in mind.

Chapter 7: KL Divergence

How to measure the difference between two distributions, and the one formula you must memorise.

7.1 The definition

The **Kullback–Leibler (KL) divergence** measures how different one distribution q is from another distribution p.

FORMULA — KL divergence (general)

$$D_{KL}(q \| p) = \int q(z) \cdot \log[q(z) / p(z)] \, dz = E_{z\sim q}[\log q(z) - \log p(z)]$$

What each symbol means

q(z)	the distribution we are checking (in a VAE: the encoder's cloud)
p(z)	the reference distribution (in a VAE: the prior N(0,1))
log q(z) − log p(z)	how much more q believes in value z than p does
$E_{z\sim q}$	average this over values that q itself considers likely
‖	read as 'relative to' or 'measured against'

In plain words: go through the values that q likes, and on average ask how much more likely q thinks they are than p does. If the two distributions agree everywhere, every log-ratio is 0 and the KL is 0.

KEY IDEA

Three facts you must know:

- KL is **always ≥ 0**. It is exactly 0 only when q and p are identical.
- The more q's shape or position differs from p, the larger the KL.
- KL is **not symmetric**: $D_{KL}(q\|p)$ is usually not equal to $D_{KL}(p\|q)$. So it is a 'divergence', not a true distance.

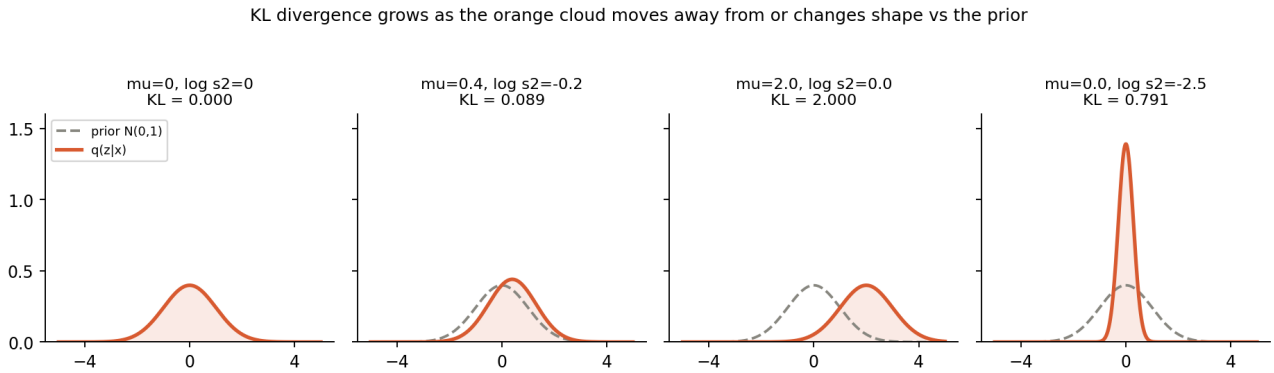


Figure 16. *Four encoder clouds (orange) against the prior (grey). Moving the centre or squeezing the width both raise the KL.*

7.2 Why asymmetry matters

$D_{KL}(q\|p)$ takes the average over places where q puts probability. So it punishes q very heavily for putting probability where p has almost none (log p becomes a huge negative number there). It barely cares if q ignores some region that p likes. This is called **mode-seeking** behaviour.

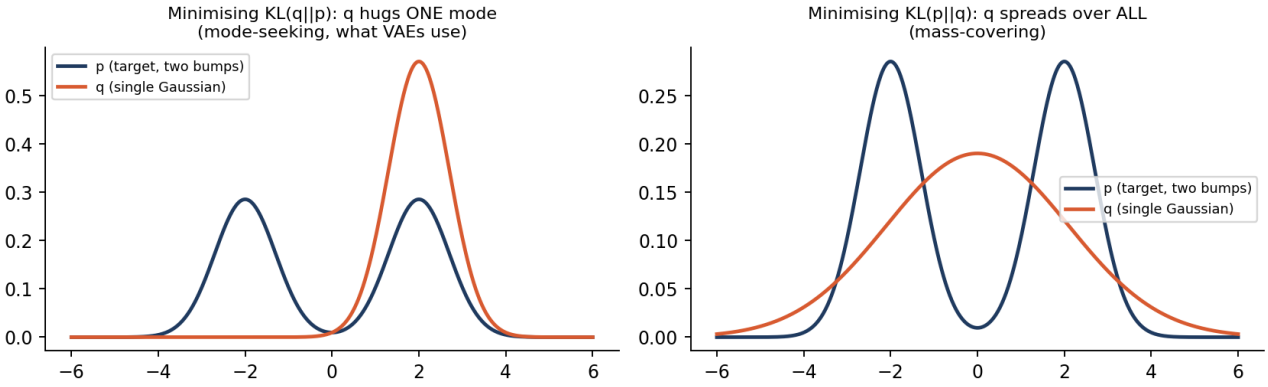


Figure 17. *When the target has two bumps, minimising KL(q‖p) makes a single Gaussian hug one bump; minimising KL(p‖q) makes it stretch over both.*

WHY DOES THIS WORK?

VAEs use $D_{KL}(q\|p)$. This discourages the encoder from placing codes in regions the prior considers very unlikely, which keeps the latent space inside the region we will later sample from.

7.3 The one formula you actually need

When q is a Gaussian $N(\mu, \sigma^2)$ and p is the standard normal $N(0, 1)$, the integral can be solved on paper once and for all. The result is a simple closed-form expression, so no integral is ever computed during training.

FORMULA — KL between N(μ, σ²) and N(0, 1), per latent dimension

$$D_{KL}(N(\mu, \sigma^2) \,\|\, N(0, 1)) = \tfrac{1}{2} \cdot (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$$

For a d-dimensional code: $D_{KL} = \tfrac{1}{2} \cdot \Sigma_{j=1..d} (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$

What each symbol means

μ^2	grows when the centre moves away from 0; pulls every cloud back to the middle
σ^2	grows when the cloud is wide; stops clouds from spreading too much
$-\log \sigma^2$	grows when the cloud is narrow; stops clouds shrinking into single points
-1	a constant that makes the total exactly 0 at $\mu = 0$, $\sigma = 1$
$\tfrac{1}{2}$	comes from the Gaussian formula; it just scales everything
Σ_j	add the KL of each latent dimension, because the dimensions are independent

In plain words: the penalty is zero only when the encoder's cloud is exactly the standard bell curve. Moving the centre costs $\mu^2/2$. Making the width too big or too small also costs, with the cheapest width being exactly 1.

Why it is built this way: the σ^2 term and the $-\log \sigma^2$ term pull in opposite directions and balance exactly at $\sigma^2 = 1$. Without $-\log \sigma^2$ the encoder would shrink every cloud to a point and we would be back to a plain autoencoder.

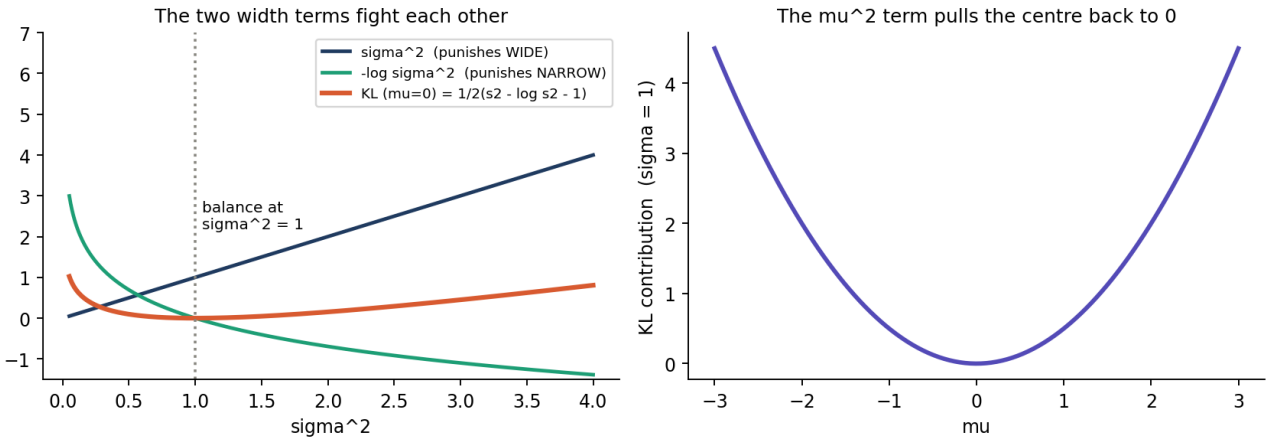


Figure 18. *Left: the two width terms fight, and the KL (orange) is lowest at $\sigma^2 = 1$. Right: the μ^2 term is a bowl centred at 0.*

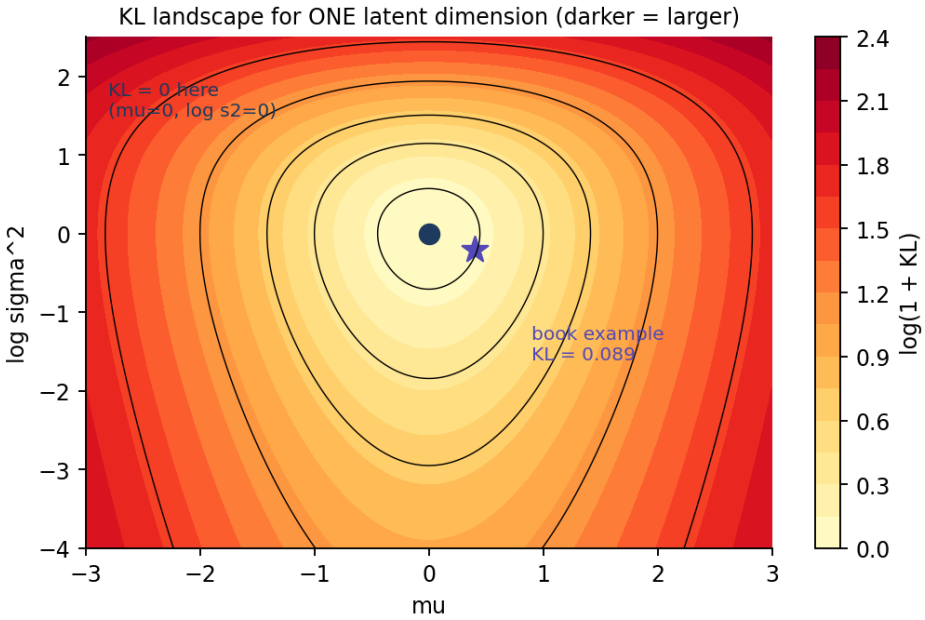


Figure 19. *The whole KL landscape for one dimension. The only point with zero cost is μ = 0, log σ² = 0.*

WORKED EXAMPLE

Compute the KL for one dimension with μ = 1.0 and log σ² = 0:

- log σ² = 0 means σ² = e^0 = 1.
- KL = ½ × (1.0² + 1 − 0 − 1) = ½ × 1 = **0.5**.
- Only the centre is wrong (it sits at 1 instead of 0), so only the μ² term contributes.

IMPORTANT - FOCUS HERE

Memorise ½(μ² + σ² − log σ² − 1). This single line is the entire regularisation loss of a VAE, and it appears directly in the training code in Chapter 12 (written with log σ² instead of σ²).

Chapter 8: Bayes' Rule and Latent Variable Models

Prior, likelihood, posterior, evidence, and why the evidence cannot be computed.

8.1 Bayes' rule

FORMULA — Bayes' rule

$$p(z \mid x) = p(x \mid z) \cdot p(z) / p(x)$$

What each symbol means

p(z)	the **prior**: what we believe about z before seeing any data. In a VAE: N(0, 1)
p(x | z)	the **likelihood**: given a code z, how likely is this x? This is what the **decoder** models
p(z | x)	the **posterior**: after seeing x, which codes are likely? This is what the **encoder** tries to approximate
p(x)	the **evidence**: the overall probability of x, averaged over every possible z. The troublemaker

In plain words: your updated belief about the hidden cause z equals how well z explains the data, times how plausible z was to begin with, divided by a normalising number that makes everything add up to 1.

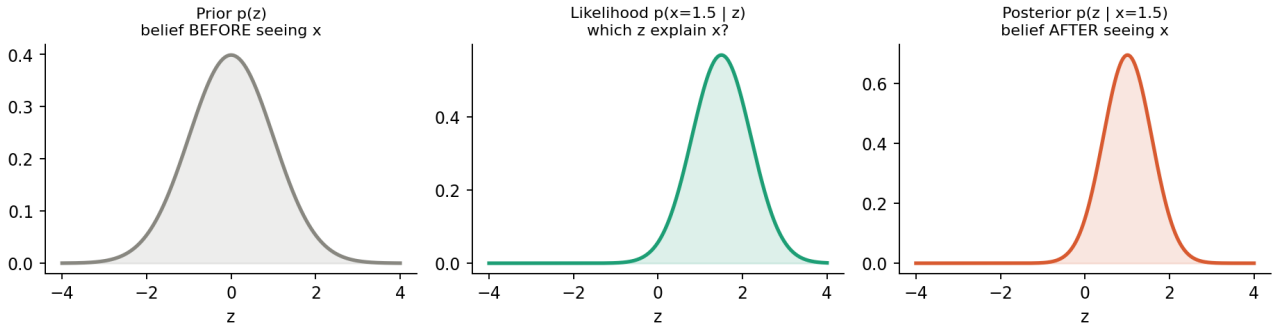


Figure 20. *Bayes' rule in pictures: the prior (grey) times the likelihood (green) gives the posterior (orange), which sits between what we believed and what the data says.*

SIMPLE ANALOGY

A doctor sees a cough (x) and wonders about the cause (z). The prior is how common each illness is. The likelihood is how often each illness causes a cough. The posterior is the doctor's updated belief after hearing the cough. The evidence is simply how common coughs are overall.

8.2 Latent variable models

A **latent variable model** assumes every data point was created by a two-step random process. First a hidden code z is drawn from the prior p(z). Then the data x is produced from z by p(x | z). This is exactly the generative process we want to learn, and it mirrors the encoder/decoder split, except that every arrow is now a probability distribution.

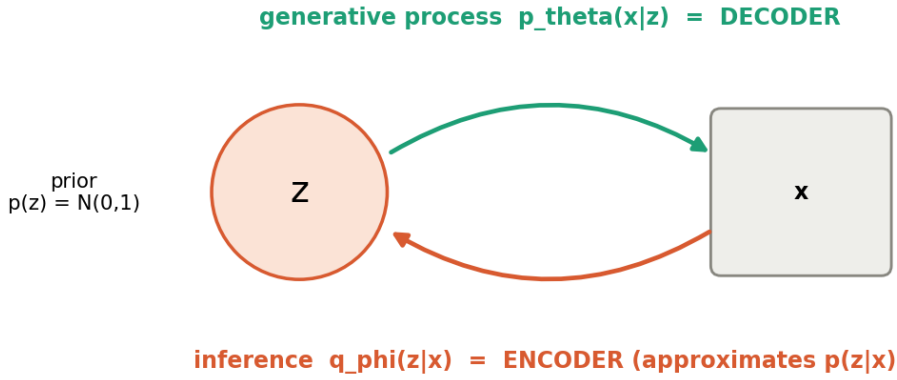


Figure 21. *The probabilistic view of a VAE. Green: generation (decoder). Orange: inference (encoder).*

8.3 The problem: p(x) is intractable

FORMULA — The evidence

$$p(x) = \int p(x \mid z) \cdot p(z)\, dz$$

What each symbol means

∫ … dz	add up over every possible value of z, which is a continuous, infinite space
p(x | z) · p(z)	how likely it is that this z was chosen AND produced x

In plain words: the probability of the data is the sum, over every possible hidden code, of 'how likely this code is' times 'how likely this code would produce x'.

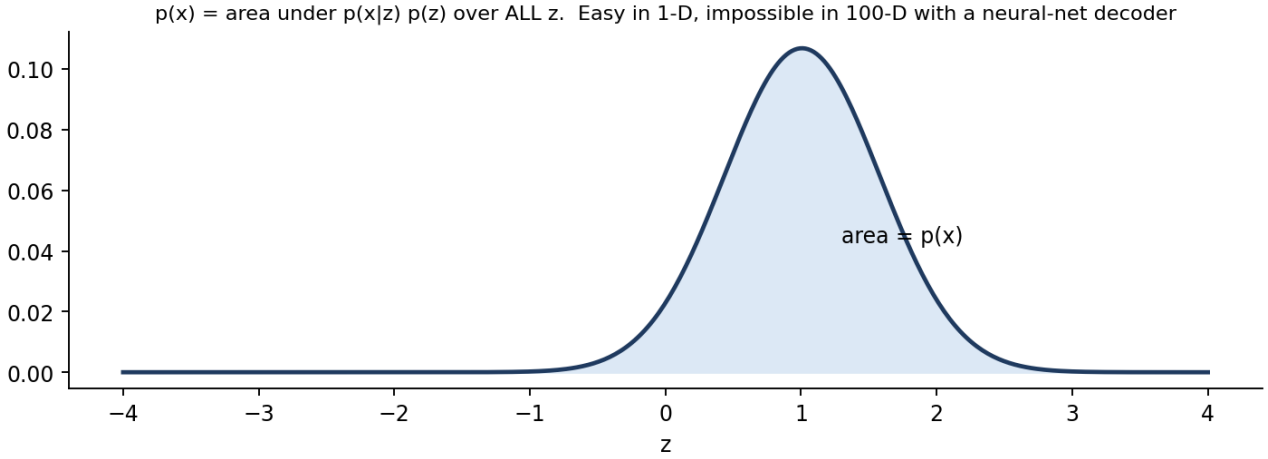


Figure 22. *In one dimension the evidence is just an area under a curve. With a 16- to 512-dimensional z and a deep neural-network decoder, no formula or grid can compute this area.*

For toy problems we could compute this. But with a neural-network decoder and many latent dimensions, the integral has no closed-form answer, and covering a

100-dimensional space with a grid is impossible (even 10 points per dimension gives 10^{100} points). And because the posterior p(z | x) divides by p(x), we cannot compute the posterior either.

IMPORTANT - FOCUS HERE

The central obstacle: we cannot compute p(x), so we cannot train by directly maximising likelihood, and we cannot compute the true posterior p(z | x). The VAE and the ELBO exist to get around exactly this obstacle.

PART III

The Variational Autoencoder: derivation, trick, code, training, and a full worked example

Chapter 9: Setting Up the Variational Problem

If we cannot compute the true posterior, we learn an approximation to it.

9.1 Restating the goal

We want a decoder $p_\theta(x \mid z)$ and a simple prior $p(z) = N(0, I)$ such that drawing z from the prior and passing it through the decoder produces realistic data. Ideally we would train by maximising the log-likelihood $\log p_\theta(x)$ of the training data. Chapter 8 showed that this needs an impossible integral.

9.2 Introducing an approximate posterior

Since the true posterior $p_\theta(z \mid x)$ cannot be computed, we introduce a separate network, the **encoder**, to learn an approximation of it. We deliberately choose the approximation to be something simple: a Gaussian whose centre and width are produced by a neural network that reads x.

FORMULA — The approximate posterior (the VAE encoder)

$$q_\varphi(z \mid x) = N(\, z \,;\, \mu_\varphi(x),\, \mathrm{diag}(\sigma_\varphi(x)^2)\,)$$

What each symbol means

$q_\varphi(z \mid x)$	'given this input x, where could its code z be?' The letter q marks it as an approximation
$\mu_\varphi(x)$	a vector of centres, one per latent dimension, computed by the encoder from x
$\sigma_\varphi(x)^2$	a vector of variances (widths squared), one per latent dimension
$\mathrm{diag}(\cdot)$	the dimensions are treated as independent: no correlations between them
φ	the encoder's weights

In plain words: for each input, the encoder does not output one code; it outputs a cloud of plausible codes described by a centre and a width in every dimension.

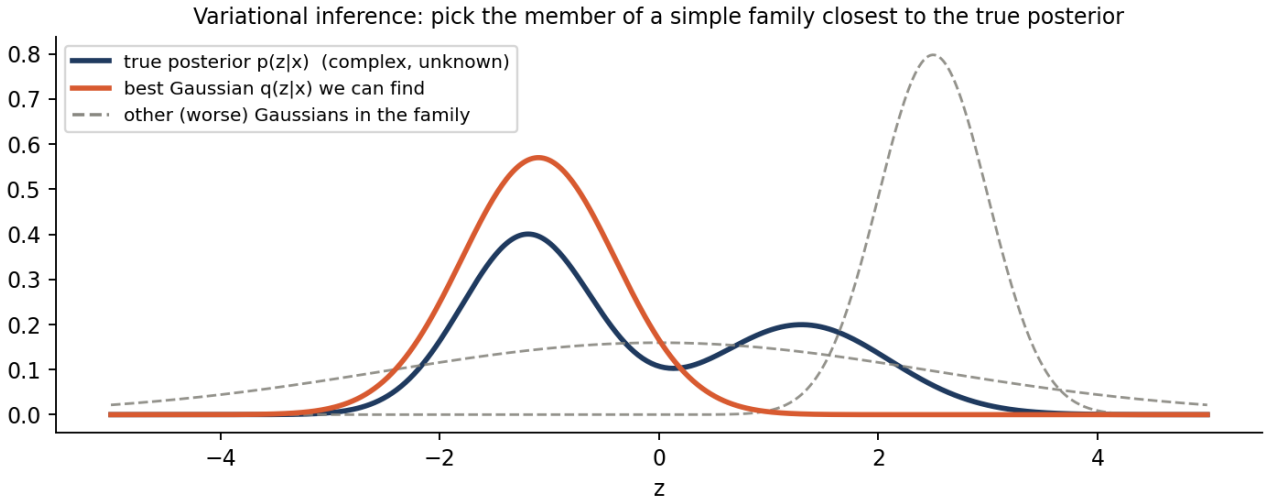


Figure 23. *Variational inference: the true posterior (navy) may be complicated. We search the family of simple Gaussians (grey dashed) for the one closest to it (orange).*

KEY IDEA

This is where the name comes from. In statistics, approximating an intractable distribution with the best member of a simpler family is called variational inference. A *Variational* Autoencoder is an autoencoder whose encoder performs variational inference. And this is exactly the 'encode to a cloud, not a point' idea from Chapter 5, now written formally.

WHY DOES THIS WORK?

Why not learn a separate q for every single training image? That is what classical variational inference does, and it is very slow. A VAE instead uses one shared network that maps any x to its q. This is called **amortised inference**: the cost of inference is 'paid in advance' during training, so at test time a single forward pass gives you the posterior for any new input.

Chapter 10: Deriving the ELBO, Step by Step

The most important equation in the book, built from nothing but algebra and Chapter 6–8.

Read this chapter slowly. Every line follows from the line before it using only three tools: multiplying by 1, Bayes' rule, and the definition of KL divergence.

10.1 Step 1: insert the encoder without changing anything

FORMULA — Step 1

$$\log p_\theta(x) = \log p_\theta(x) \cdot \int q_\varphi(z|x)\, dz = \int q_\varphi(z|x) \cdot \log p_\theta(x)\, dz = E_q[\, \log p_\theta(x)\,]$$

In plain words: the area under any distribution is 1, so multiplying by ∫ q dz changes nothing. Since log p(x) does not depend on z, we can move it inside the integral, and an integral weighted by q is just an expectation under q.

10.2 Step 2: use Bayes' rule, then add and subtract the same thing

Bayes' rule rearranged gives $p_\theta(x) = p_\theta(x, z) / p_\theta(z \mid x)$. Substitute it, split the log of a fraction into a difference of logs, and then add and subtract $\log q_\varphi(z \mid x)$ (adding zero changes nothing):

FORMULA — Step 2

$$\log p_\theta(x) = E_q[\, \log p_\theta(x,z) - \log p_\theta(z|x)\,]$$
$$= E_q[\, \log p_\theta(x,z) - \log q_\varphi(z|x) + \log q_\varphi(z|x) - \log p_\theta(z|x)\,]$$
$$= E_q[\, \log p_\theta(x,z) - \log q_\varphi(z|x)\,] + E_q[\, \log q_\varphi(z|x) - \log p_\theta(z|x)\,]$$

In plain words: the last line uses linearity of expectation to split one average into two averages.

10.3 Step 3: recognise the two pieces

Compare the second bracket with the KL definition in Chapter 7: $E_q[\log q - \log p]$. It is exactly the KL divergence between our encoder $q_\varphi(z \mid x)$ and the *true* posterior $p_\theta(z \mid x)$. The first bracket gets a name: the **Evidence Lower BOund**, or ELBO.

FORMULA — The master identity

$$\log p_\theta(x) = \mathrm{ELBO}(x) + D_{KL}(\, q_\varphi(z|x) \,\|\, p_\theta(z|x)\,)$$

$$\mathrm{ELBO}(x) = E_q[\, \log p_\theta(x, z) - \log q_\varphi(z|x)\,]$$

What each symbol means

$\log p_\theta(x)$	what we truly want to maximise (intractable)
ELBO(x)	a quantity we CAN compute, using samples from the encoder
D_{KL}(q ‖ true posterior)	the gap: how far our encoder is from the true posterior (cannot be computed)

In plain words: the thing we want equals the thing we can compute plus a gap we cannot compute.

10.4 Step 4: why this is exactly what we need

We still cannot compute the gap because it contains the true posterior. But we know one crucial fact about it: it is a KL divergence, and every KL divergence is ≥ 0. Therefore:

FORMULA — The lower bound

$$\log p_\theta(x) \geq \mathrm{ELBO}(x)$$

In plain words: the ELBO is always a floor underneath the log-likelihood. This is why it is called the Evidence Lower Bound (evidence is another name for p(x)).

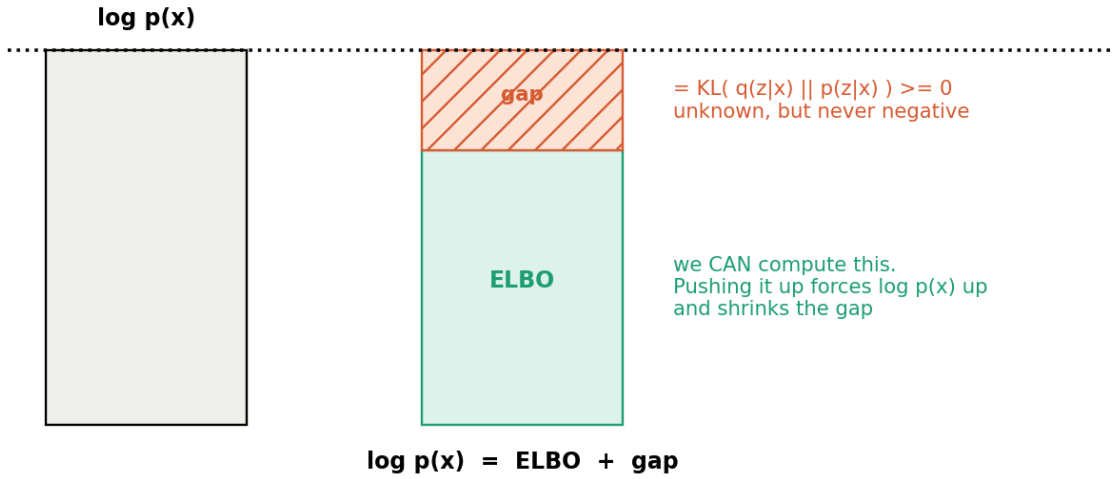


Figure 24. *log p(x) is split into the ELBO (which we can compute) and a non-negative gap (which we cannot).*

KEY IDEA

Maximising the ELBO does two good things at once:

- It pushes up a guaranteed floor under log p(x), so the generative model (the decoder) gets better.
- Because log p(x) = ELBO + gap, for a fixed decoder, raising the ELBO *must* shrink the gap, so the encoder is automatically pulled towards the true posterior.

SIMPLE ANALOGY

You want to know how tall a building is, but you cannot measure the roof. You can measure a platform that is guaranteed to be at or below the roof. By pushing the platform as high as possible, you both learn more about the roof height and close the distance between the platform and the roof.

10.5 Step 5: split the ELBO into two practical terms

The ELBO still contains the joint $p_\theta(x, z)$. Write it as $p_\theta(x \mid z) \cdot p(z)$ (decoder times prior), split the log again, and regroup:

FORMULA — The practical ELBO

$$\begin{aligned} \text{ELBO}(x) &= E_q[\ \log p_\theta(x|z) + \log p(z) - \log q_\varphi(z|x)\ ] \\ &= E_q[\ \log p_\theta(x|z)\ ] - E_q[\ \log q_\varphi(z|x) - \log p(z)\ ] \\ &= E_q[\ \log p_\theta(x|z)\ ] - D_{KL}(\ q_\varphi(z|x) \parallel p(z)\ ) \end{aligned}$$

What each symbol means

$E_q[\ \log p_\theta(x\|z)\ ]$	**reconstruction term**: sample z from the encoder; how well does the decoder rebuild x?
$D_{KL}(\ q_\varphi(z\|x) \parallel p(z)\ )$	**regularisation term**: how far is the encoder's cloud from the prior N(0,1)?

In plain words: a good model rebuilds the input well, while keeping every input's cloud close to the standard bell curve.

Why it is built this way: notice the KL here is against the *prior* p(z), which is known, not against the unknown true posterior. So this KL has the closed form from Chapter 7. There is no intractable integral left anywhere.

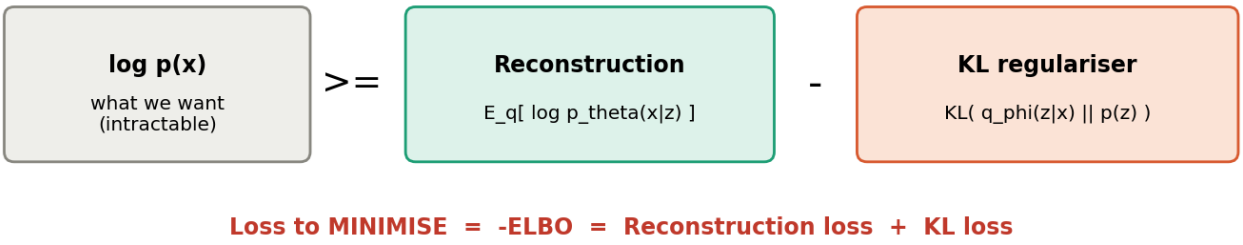


Figure 25. *The ELBO = reconstruction minus KL. The training loss is its negative.*

10.6 From ELBO to the loss you code

PyTorch minimises, so we flip the sign. How does the reconstruction term become MSE or BCE? If the decoder models each pixel as a Bernoulli (on/off) variable with probability $\hat{x}_i$, then $-\log p_\theta(x|z)$ is exactly the BCE loss. If it models each value as a Gaussian centred at $\hat{x}_i$ with fixed variance, then $-\log p_\theta(x|z)$ equals the squared error times a constant, plus another constant. So the familiar losses from Chapter 3 are really log-likelihoods in disguise.

FORMULA — The VAE loss

$$L_{VAE}(x) = -\text{ELBO}(x) = \text{Reconstruction_Loss}(x, \hat{x}) + D_{KL}(\, N(\mu, \sigma^2) \,\|\, N(0, 1)\,)$$
$$= \|x - \hat{x}\|^2 + \tfrac{1}{2} \cdot \Sigma_j \,(\, \mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1\,)$$

In plain words: the loss is: how badly we rebuilt the input, plus how far each latent cloud strays from the standard bell curve. Every symbol in this line has now been derived; nothing is hidden.

COMMON MISTAKE

Thinking the KL in the loss is the same as the gap. They are different KLs. The loss uses KL(q ‖ prior), which we can compute. The gap is KL(q ‖ true posterior), which we never compute; we only know it is ≥ 0.

IMPORTANT - FOCUS HERE

Strictly, the MSE form corresponds to a Gaussian decoder with variance ½. If your data variance is very different, the balance between reconstruction and KL changes. This is one hidden reason why the β weight in Chapter 17 matters in practice.

Chapter 11: The Reparameterisation Trick

One line of algebra that makes the whole VAE trainable by ordinary backpropagation.

11.1 The problem: sampling is not differentiable

To train the encoder, gradients must flow from the loss, through the sampled z, back into μ and σ, and from there into the encoder weights φ. But z is produced by a random draw $z \sim N(\mu, \sigma^2)$. You cannot ask ordinary calculus 'how does this random draw change if I nudge μ?' because a dice roll has no derivative.

11.2 The fix: move the randomness outside

FORMULA — The reparameterisation trick

$$\varepsilon \sim N(0, 1) \quad z = \mu + \sigma \cdot \varepsilon$$

What each symbol means

ε (epsilon)	pure random noise from the standard bell curve. It has no learnable parameters
σ · ε	stretches the noise to the right width
μ + …	shifts it to the right centre
z	has exactly the same distribution $N(\mu, \sigma^2)$ as before, but is now built from plain arithmetic

In plain words: instead of drawing directly from the encoder's bell curve, draw from the standard bell curve and then stretch and shift the result. You get the same kind of sample, but now z is an ordinary function of μ and σ.

Why it is built this way: the derivatives become trivial: $\partial z/\partial \mu = 1$ and $\partial z/\partial \sigma = \varepsilon$. So the chain rule can pass straight through, exactly like any other layer. The randomness sits in ε, which we never need to differentiate.

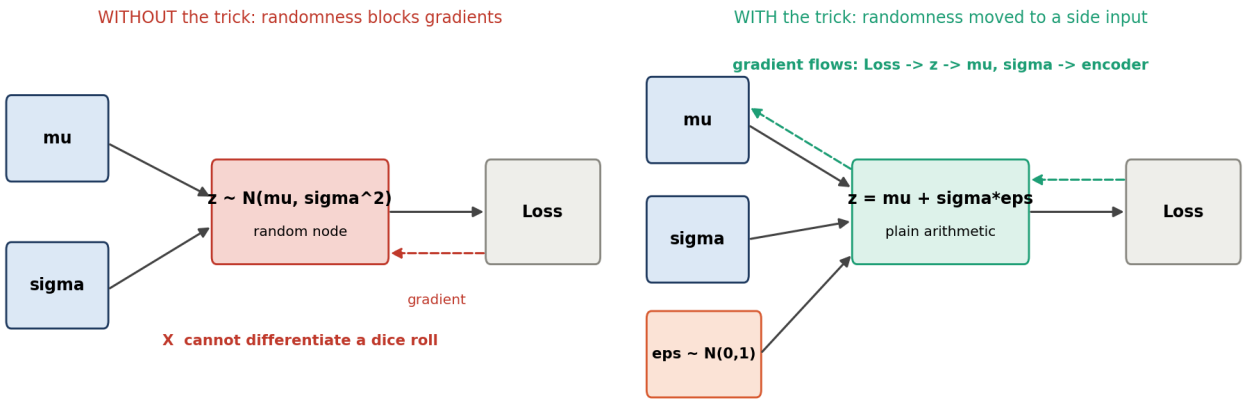


Figure 26. *Left: a random node blocks the gradient. Right: the randomness enters as a side input, and gradients flow back to μ and σ.*

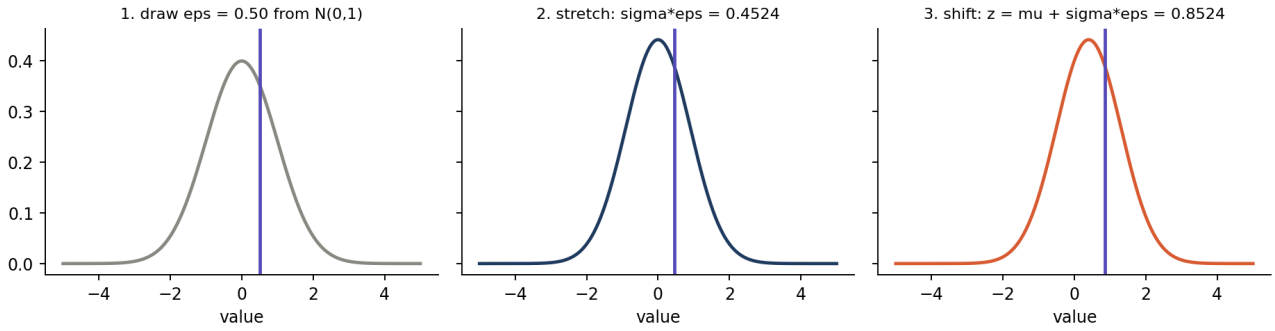


Figure 27. *The trick in three pictures, using the worked-example numbers: draw ε, stretch by σ, shift by μ.*

SIMPLE ANALOGY

Instead of asking a machine 'give me a random height for a person in this class', you roll a standard die yourself and then compute 'average height + spread × die result'. The randomness is still there, but the recipe that turns it into a height is a clear formula you can adjust and differentiate.

WORKED EXAMPLE

Using μ = [0.40, −0.30], σ = [0.9048, 1.0513] and ε = [0.50, −1.20]:

- $z_1 = 0.40 + 0.9048 \times 0.50 = 0.40 + 0.4524 =$ **0.8524**
- $z_2 = -0.30 + 1.0513 \times (-1.20) = -0.30 - 1.2616 =$ **−1.5616**
- Gradient example: if the loss changes by 2.0 per unit change of z_1, then it changes by $2.0 \times 1 = 2.0$ per unit of μ_1, and by $2.0 \times \varepsilon_1 = 2.0 \times 0.5 = 1.0$ per unit of σ_1.

11.3 Why not use another gradient method?

There is an alternative called the **score-function** or **REINFORCE** estimator, which works even when you cannot reparameterise. But its estimates are much noisier. The figure below is not a sketch: it measures, by simulation, the spread of both estimators for the same simple problem.

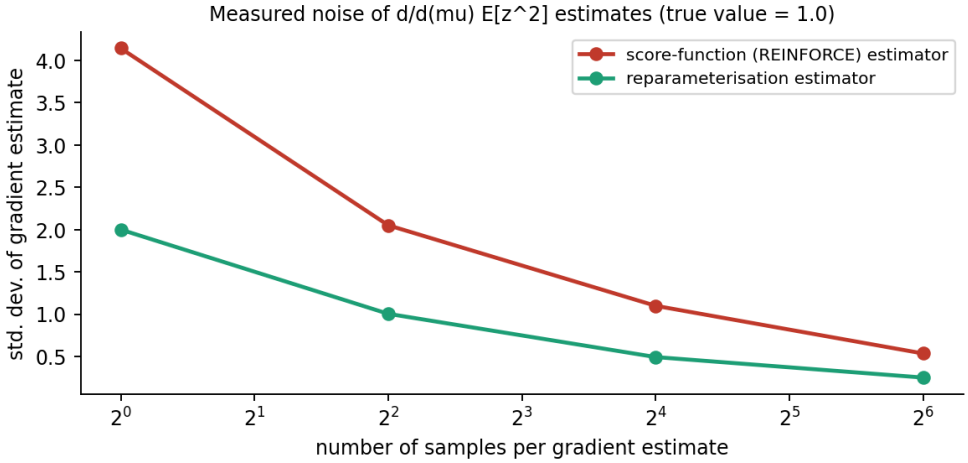


Figure 28. *Measured noise of two gradient estimators for the same quantity. The reparameterisation estimator (green) is less noisy at every sample size; in real high-dimensional VAEs the gap is usually far larger.*

KEY IDEA

The reparameterisation trick turns VAE training into ordinary feed-forward training. Without it, VAEs would need slow, high-variance estimators. With it, one sample of ε per input is usually enough.

Chapter 12: Full VAE Architecture and Implementation

Every box, every number, every line of code, and how they map to the maths.

12.1 The complete picture

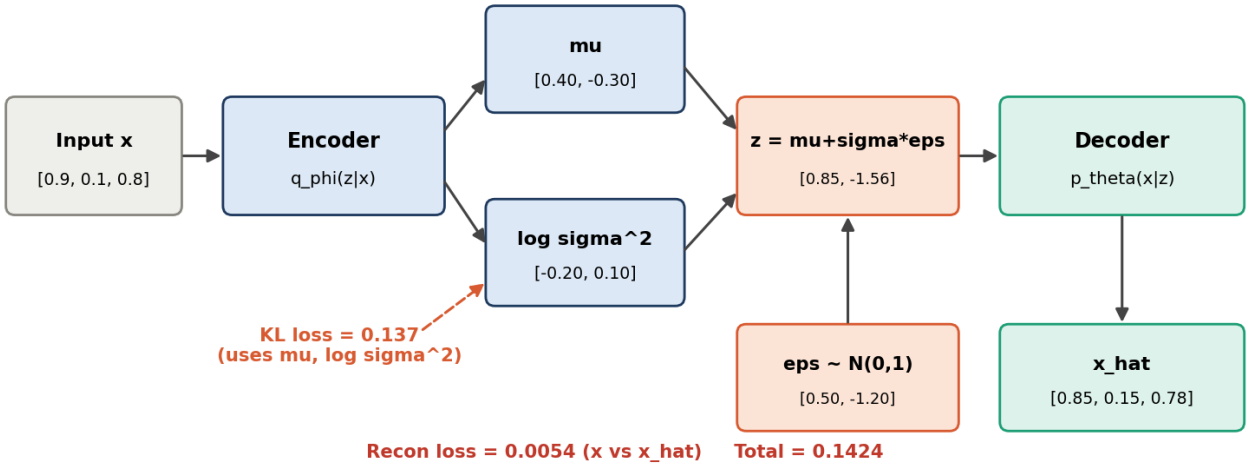


Figure 29. *The complete VAE forward pass with the numbers from the worked example in Chapter 14.*

Compare this with the plain autoencoder (Chapter 3). The architecture is almost identical, with exactly two differences: (1) the encoder outputs two vectors, μ and log σ^2, instead of one; and (2) there is a sampling step between encoder and decoder. Everything else you know about training networks still applies.

12.2 Why the encoder outputs log σ^2

A variance must always be positive, but the last layer of a network can output any real number, including negative ones. If the network outputs log σ^2 instead, every real number is a valid answer, and we recover the width by exponentiating, which always gives a positive result.

FORMULA — From log-variance to standard deviation

$$\sigma = \exp(\tfrac{1}{2} \cdot \log \sigma^2) \quad \sigma^2 = \exp(\log \sigma^2)$$

What each symbol means

$\log \sigma^2$	raw encoder output, any real number
exp	the exponential function; always positive
½	because $\sigma = \sqrt{(\sigma^2)}$, and a square root becomes ½ inside a log

In plain words: take the network's output, halve it, and exponentiate to get the standard deviation.

Why it is built this way: besides guaranteeing positivity, working in log-space is numerically stable: very small and very large widths become moderate numbers like −5 or 3, which gradient descent handles well.

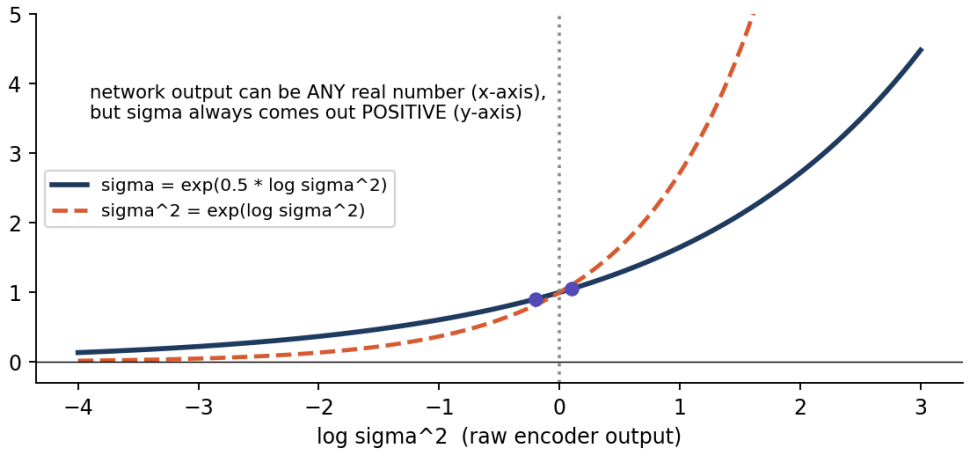


Figure 30. *Any value on the horizontal axis is allowed, yet σ on the vertical axis is always positive.*

12.3 Full PyTorch implementation

PyTorch: a complete VAE for 28x28 images

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class VAE(nn.Module):
    def __init__(self, latent_dim=16):
        super().__init__()
        self.enc_trunk = nn.Sequential(                 # shared encoder body
            nn.Flatten(),
            nn.Linear(28*28, 256), nn.ReLU(),
            nn.Linear(256, 64),    nn.ReLU(),
        )
        self.fc_mu     = nn.Linear(64, latent_dim)      # head 1: centre  mu
        self.fc_logvar = nn.Linear(64, latent_dim)      # head 2: log sigma^2
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 64), nn.ReLU(),
            nn.Linear(64, 256),        nn.ReLU(),
            nn.Linear(256, 28*28),     nn.Sigmoid(),
            nn.Unflatten(1, (1, 28, 28))
        )

    def encode(self, x):
        h = self.enc_trunk(x)
        return self.fc_mu(h), self.fc_logvar(h)

    def reparameterize(self, mu, logvar):
        std = torch.exp(0.5 * logvar)       # sigma = exp(0.5 * log sigma^2)
        eps = torch.randn_like(std)         # eps ~ N(0, 1)
        return mu + std * eps               # z = mu + sigma * eps

    def forward(self, x):
        mu, logvar = self.encode(x)
        z = self.reparameterize(mu, logvar)
        return self.decoder(z), mu, logvar

def vae_loss(x, x_hat, mu, logvar):
    recon = F.binary_cross_entropy(x_hat, x, reduction="sum")
    kl = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
    return recon + kl, recon, kl

model = VAE(latent_dim=16)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
for x, _ in dataloader:
    x_hat, mu, logvar = model(x)
    loss, recon, kl = vae_loss(x, x_hat, mu, logvar)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

12.4 Mapping every formula to a line of code

Maths	Code	Chapter
μ, $\log \sigma^2$ = encoder heads	self.fc_mu(h), self.fc_logvar(h)	9, 12
$\sigma = \exp(\tfrac{1}{2} \log \sigma^2)$	std = torch.exp(0.5 * logvar)	12
$\varepsilon \sim N(0, 1)$	eps = torch.randn_like(std)	11
$z = \mu + \sigma\varepsilon$	return mu + std * eps	11
$-E_q[\log p_\theta(x\|z)]$	F.binary_cross_entropy(..., reduction='sum')	10
$\tfrac{1}{2} \Sigma(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$	-0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())	7

Maths	Code	Chapter
−ELBO	recon + kl	10

WHY DOES THIS WORK?

Check the KL line: $-0.5 \times (1 + \log \sigma^2 - \mu^2 - \sigma^2)$ is the same as $\frac{1}{2} \times (\mu^2 + \sigma^2 - \log \sigma^2 - 1)$. Multiplying by -1 flips every sign inside. It is written this way only for convenience; the maths is identical.

IMPORTANT - FOCUS HERE

Use the **same reduction** for both terms. The derivation sums over pixels and over latent dimensions. If you use a *mean* for the reconstruction (divided by 784) but a *sum* for the KL, the KL becomes about 784 times stronger than intended, and you will likely get blurry outputs or posterior collapse.

12.5 Generating new samples after training

Generation: exactly the 'tempting idea' from Chapter 5, which now works

```
model.eval()
with torch.no_grad():
    z = torch.randn(16, latent_dim)      # 16 codes sampled from the prior N(0, I)
    samples = model.decoder(z)            # 16 brand-new images
```

KEY IDEA

At generation time the encoder is not used at all. You only need the prior and the decoder. The encoder's job during training was to shape the latent space so that the prior is a safe place to sample from.

Chapter 13: Training Dynamics and Common Failure Modes

What healthy training looks like, the tug-of-war between the two losses, and posterior collapse.

13.1 What healthy training looks like

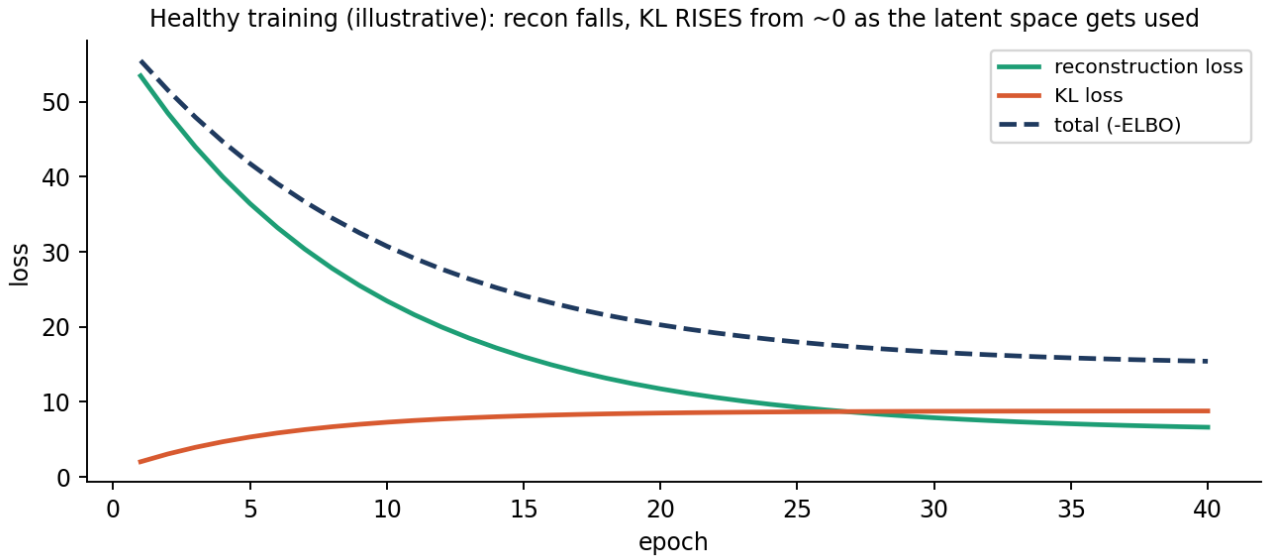


Figure 31. *Typical (illustrative) VAE training curves. The KL term starts near zero and rises.*

Early in training the encoder has learned nothing, so it tends to output $\mu \approx 0$ and $\sigma \approx 1$ for every input. That matches the prior exactly, so the KL is almost zero, but the reconstructions are blurry and generic because z carries no information about the input. As training continues, the encoder begins placing different inputs in different regions to improve reconstruction, and the KL rises: this rise is a sign that the latent code is being used. A healthy run settles into a balance where both terms are clearly above zero.

13.2 The reconstruction versus regularisation tug-of-war

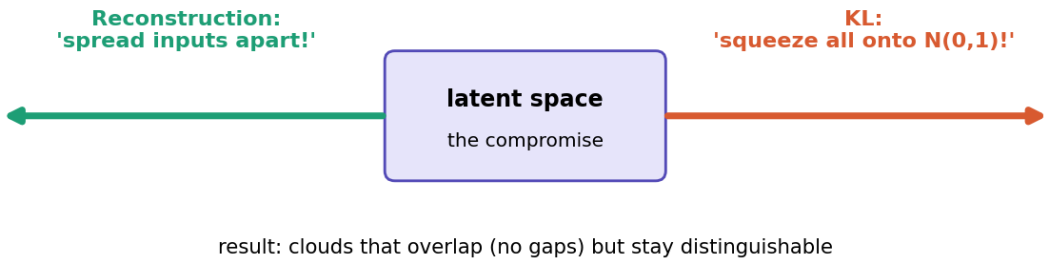


Figure 32. *The two loss terms pull in opposite directions; the final latent space is the compromise.*

The reconstruction term wants each input's cloud to be small and far from others, so that inputs are easy to tell apart. The KL term wants every cloud to be exactly N(0, 1), which would make all inputs identical. Neither wins completely. The compromise is a latent

space of overlapping but distinguishable clouds: exactly the continuous, gap-free space that makes generation possible.

KEY IDEA
Information view: the KL term is a **price in nats** (natural-log units of information) for every bit of information the code carries about x. The encoder only 'pays' for information that improves reconstruction enough to be worth the price. This is why unimportant details get dropped.

13.3 Posterior collapse

Posterior collapse is a common failure, especially with a very powerful decoder (for example, an autoregressive decoder that can predict each pixel from the previous ones) or with a KL weight that is too strong. The decoder learns to produce average-looking outputs without using z at all. The encoder then finds it cheapest to output $\mu = 0$ and $\sigma = 1$ for every input, because that makes the KL zero. The latent code stops carrying any information.

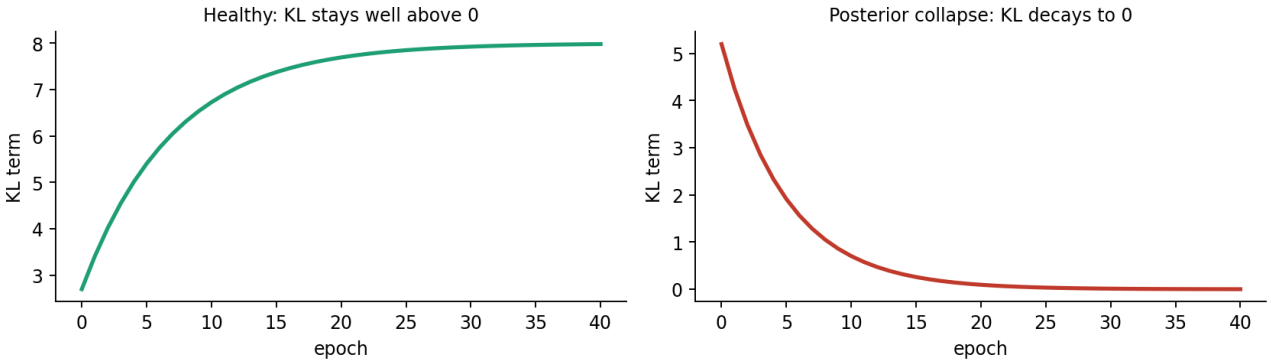


Figure 33. *Healthy KL (left) versus collapse (right), where KL decays to zero.*

IMPORTANT - FOCUS HERE
How to spot collapse:

- The average KL per batch is close to 0 for the whole run.
- Different inputs give almost identical μ values.
- Changing z at the decoder input barely changes the output.

13.4 Common fixes

KL annealing (warm-up). Multiply the KL by a weight β that starts at 0 and slowly rises to 1. The model first learns to reconstruct and to use z, and only later has to pay the full price for it. Cyclical annealing repeats this ramp several times.

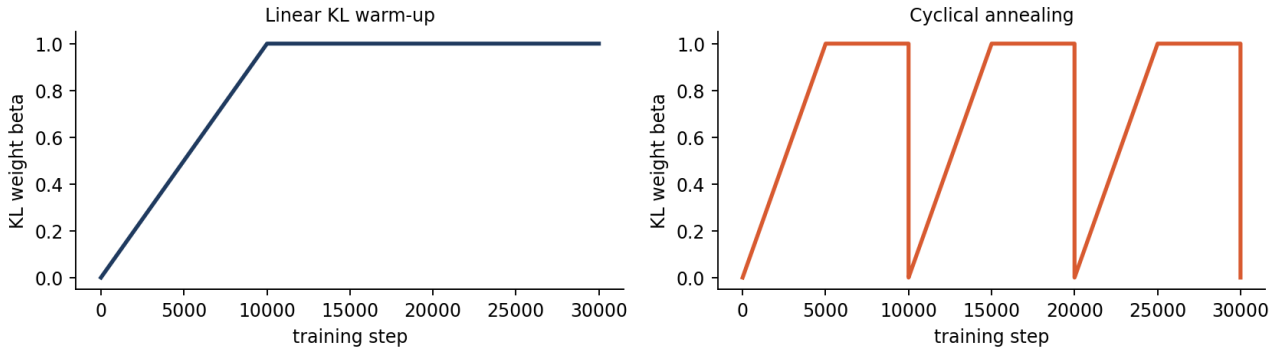


Figure 34. *Two common schedules for the KL weight.*

Free bits. Only penalise the KL of a latent dimension when it goes above a small threshold λ. Below the threshold the dimension is free to carry information, so the encoder has no reason to push it all the way to zero.

FORMULA — Free bits

$$L_{KL} = \Sigma_j \max(\lambda, KL_j)$$

What each symbol means

KL_j	KL of latent dimension j (usually averaged over the batch)
λ	the free allowance, e.g. 0.1 to 0.5 nats

In plain words: each dimension gets a small free budget of information; only what goes beyond it is charged.

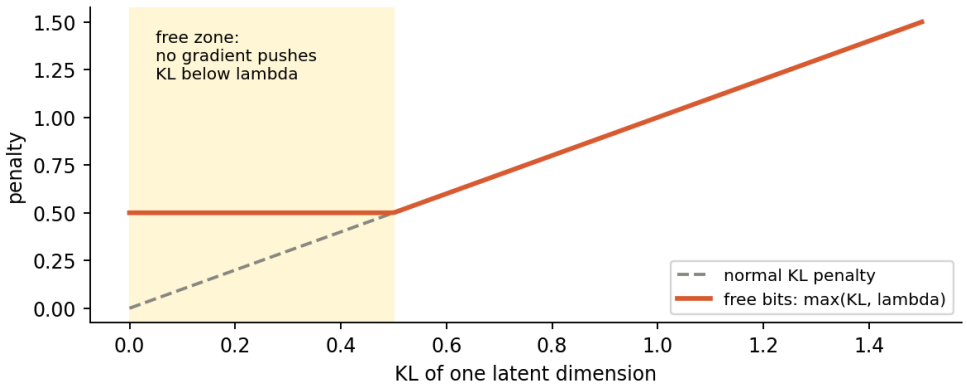


Figure 35. *With free bits, the penalty is flat below λ, so there is no gradient pushing that dimension to zero.*

β below 1. A weight $\beta < 1$ on the KL (Chapter 17) lowers the price of information and reduces collapse, at some cost to how regular the latent space is. **Weaker decoders.** Limiting the decoder's context or capacity prevents it from being powerful enough to ignore z in the first place.

Chapter 14: A Complete Worked Numerical Example

Every quantity from Chapters 9–12 computed by hand for one tiny input.

Equations are easiest to trust once you have pushed real numbers through them. Here a 3-number input is encoded into a 2-dimensional latent code. Check every line with a calculator.

14.1 Setup

Input x = [0.9, 0.1, 0.8], already scaled to [0, 1] like three pixel intensities. After the encoder trunk, the two heads output:

FORMULA — Encoder outputs

$$\mu = [\ 0.40, -0.30\ ] \quad \log \sigma^2 = [\ -0.20, 0.10\ ]$$

14.2 Recover the widths

STEP 1 — WIDTHS

- $\sigma_1 = \exp(0.5 \times -0.20) = \exp(-0.10) \approx \mathbf{0.9048}$
- $\sigma_2 = \exp(0.5 \times 0.10) = \exp(0.05) \approx \mathbf{1.0513}$
- Variances: $\sigma_1^2 = \exp(-0.20) \approx 0.8187$ and $\sigma_2^2 = \exp(0.10) \approx 1.1052$

14.3 Sample z with the reparameterisation trick

STEP 2 — SAMPLE

Suppose the noise drawn is ε = [0.50, −1.20].

- $z_1 = 0.40 + 0.9048 \times 0.50 = 0.40 + 0.4524 = \mathbf{0.8524}$
- $z_2 = -0.30 + 1.0513 \times (-1.20) = -0.30 - 1.2616 = \mathbf{-1.5616}$
- So z = [0.8524, −1.5616]. Note z_2 is far from its centre because $\varepsilon_2 = -1.2$ is a fairly unlikely draw.

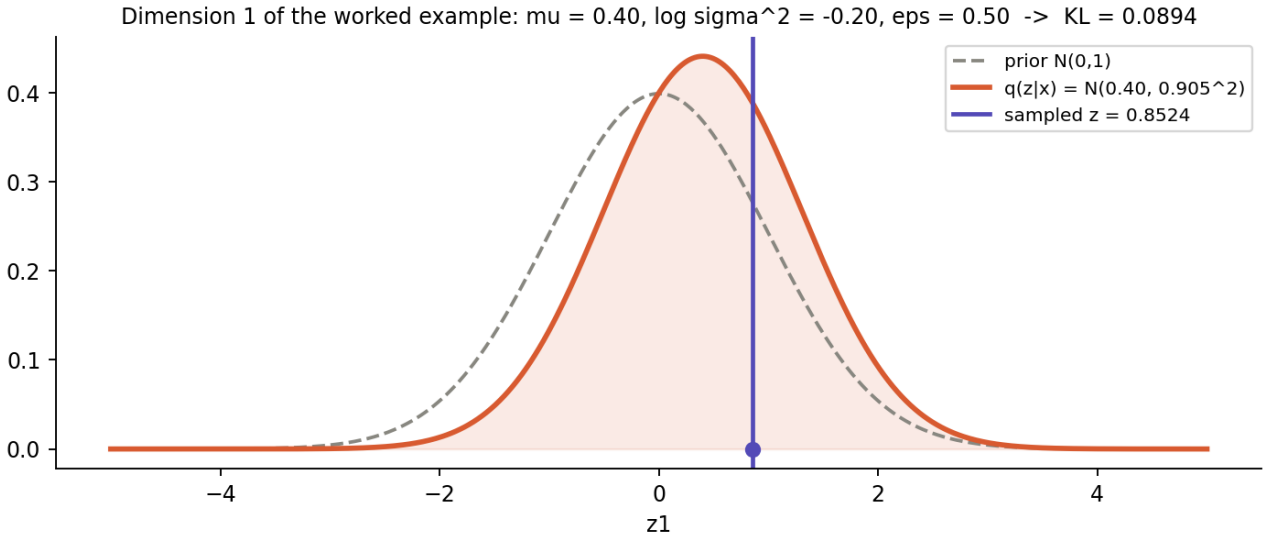


Figure 36. *Dimension 1 of this example, drawn like the interactive explorer: the encoder's cloud (orange), the prior (grey dashed) and the sampled z (purple line).*

14.4 Reconstruction loss

STEP 3 — RECONSTRUCTION

The decoder turns z into $\hat{x}$ = [0.85, 0.15, 0.78]. Using sum-of-squares:

- $(0.90 - 0.85)^2 = 0.05^2 = 0.0025$
- $(0.10 - 0.15)^2 = (-0.05)^2 = 0.0025$
- $(0.80 - 0.78)^2 = 0.02^2 = 0.0004$
- Reconstruction = 0.0025 + 0.0025 + 0.0004 = **0.0054**

14.5 KL term

STEP 4 — KL

Using $\frac{1}{2}(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$ per dimension:

- Dimension 1: $\frac{1}{2} \times (0.16 + 0.8187 - (-0.20) - 1) = \frac{1}{2} \times 0.1787$ = **0.0894**
- Dimension 2: $\frac{1}{2} \times (0.09 + 1.1052 - 0.10 - 1) = \frac{1}{2} \times 0.0952$ = **0.0476**
- Total KL = 0.0894 + 0.0476 = **0.1370**
- Reading it: dimension 1 costs more because its centre (0.40) is further from 0 than dimension 2's (−0.30) and its width is further from 1.

14.6 The final loss

FORMULA — Total

$$L_{VAE} = 0.0054 + 0.1370 = 0.1424$$

In plain words: this single number is backpropagated through the decoder, through $z = \mu + \sigma\varepsilon$, and into the encoder.

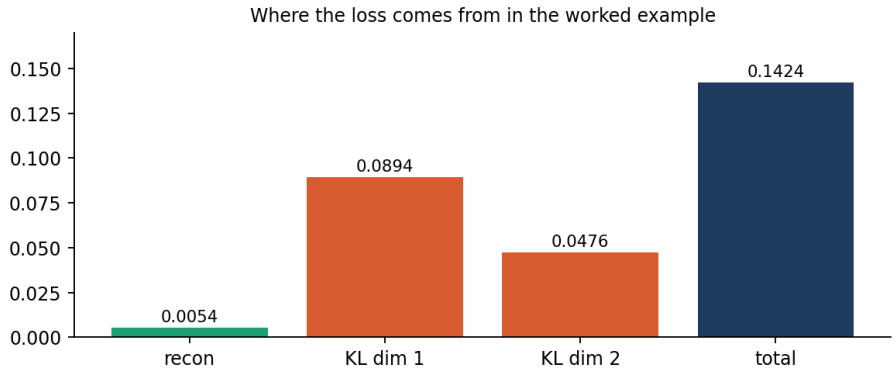


Figure 37. *Where the loss comes from. Here the KL dominates, which is normal early in training with tiny toy inputs.*

WHY DOES THIS WORK?

The KL (0.137) is much larger than the reconstruction (0.0054) only because this toy input has 3 numbers. For a 784-pixel image the reconstruction sum is much bigger, and the two terms are usually closer in size.

TRY IT YOURSELF

Recompute everything with $\mu = [0, 0]$ and $\log \sigma^2 = [0, 0]$. You should get $\sigma = [1, 1]$ and a KL of exactly 0. This is also a great unit test for your own code: feed zeros to your KL function and assert the result is 0.

Chapter 15: Convolutional VAEs for Real Image Data

Swapping the layers while keeping the whole VAE framework unchanged.

15.1 Why flatten-and-linear does not scale

The MLP VAE works for 28×28 digits, but flattening a 256×256×3 image into one vector of 196,608 numbers throws away all spatial structure (which pixels are neighbours) and needs enormous weight matrices. Real image VAEs use convolutional layers in the encoder and transposed-convolution (or upsampling) layers in the decoder.

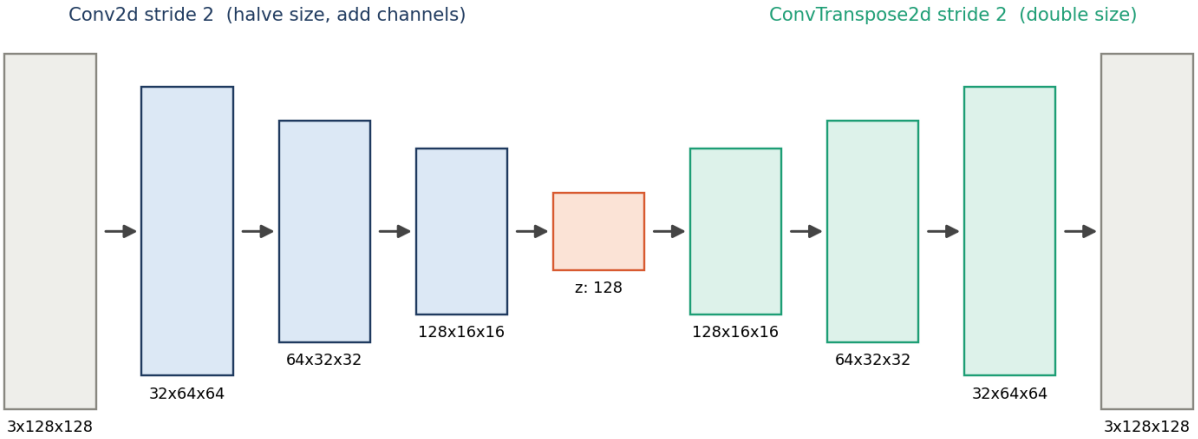


Figure 38. *Shapes through a convolutional VAE: each stride-2 convolution halves the height and width and adds channels; the decoder reverses this.*

15.2 Code sketch

PyTorch: convolutional VAE for 3x128x128 images

```
class ConvVAE(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.enc = nn.Sequential(
            nn.Conv2d(3, 32, 4, stride=2, padding=1),  nn.ReLU(),   # 128 -> 64
            nn.Conv2d(32, 64, 4, stride=2, padding=1), nn.ReLU(),   # 64 -> 32
            nn.Conv2d(64, 128, 4, stride=2, padding=1), nn.ReLU(),  # 32 -> 16
            nn.Flatten())
        self.fc_mu     = nn.Linear(128*16*16, latent_dim)
        self.fc_logvar = nn.Linear(128*16*16, latent_dim)
        self.fc_dec    = nn.Linear(latent_dim, 128*16*16)
        self.dec = nn.Sequential(
            nn.Unflatten(1, (128, 16, 16)),
            nn.ConvTranspose2d(128, 64, 4, stride=2, padding=1), nn.ReLU(),  # 16 -> 32
            nn.ConvTranspose2d(64, 32, 4, stride=2, padding=1),  nn.ReLU(),  # 32 -> 64
            nn.ConvTranspose2d(32, 3, 4, stride=2, padding=1),   nn.Sigmoid()) # 64 -> 128
    def encode(self, x):
        h = self.enc(x); return self.fc_mu(h), self.fc_logvar(h)
    def decode(self, z):
        return self.dec(self.fc_dec(z))
    def forward(self, x):
        mu, logvar = self.encode(x)
        z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
        return self.decode(z), mu, logvar
```

FORMULA — Output size of a convolution

$$\text{out} = \lfloor (\text{in} + 2\cdot\text{padding} - \text{kernel}) / \text{stride} \rfloor + 1$$

What each symbol means

in	input height or width
kernel, stride, padding	the layer settings
⌊·⌋	round down

In plain words: with kernel 4, stride 2, padding 1: out = (in + 2 − 4)/2 + 1 = in/2. So every such layer halves the size, e.g. 128 → 64.

KEY IDEA

The VAE framework is architecture-agnostic. The loss, the reparameterisation trick and the training loop are unchanged. Only the encoder and decoder internals differ. You can swap in convolutions, Transformer blocks, PointNet layers or graph networks to suit your data.

15.3 Practical tips

- Use strided convolutions rather than pooling for downsampling, so the reduction is learned.
- Keep encoder and decoder symmetric so the spatial sizes line up on the way back.
- Batch or Group Normalisation between conv layers usually stabilises training.
- For high resolution, compress to a *spatial* latent grid (for example 64×64×4) instead of one flat vector. This is exactly what Stable Diffusion's VAE does (Chapter 19).

PART IV

The VAE family and its neighbours

Chapter 16: Conditional VAE (CVAE)

Controlling what the model generates.

16.1 The limitation of a plain VAE

A plain VAE lets you generate a random, plausible sample, but you cannot choose what you get. Trained on digits 0–9, sampling z ~ N(0, 1) might give a 7 or a 3, and there is no way to ask for a specific digit.

16.2 The fix: condition everything on a label

The Conditional VAE feeds an extra piece of information y (a class label, an attribute, a text embedding, a robot's goal position) into **both** the encoder and the decoder, usually by concatenating it to the input of each.

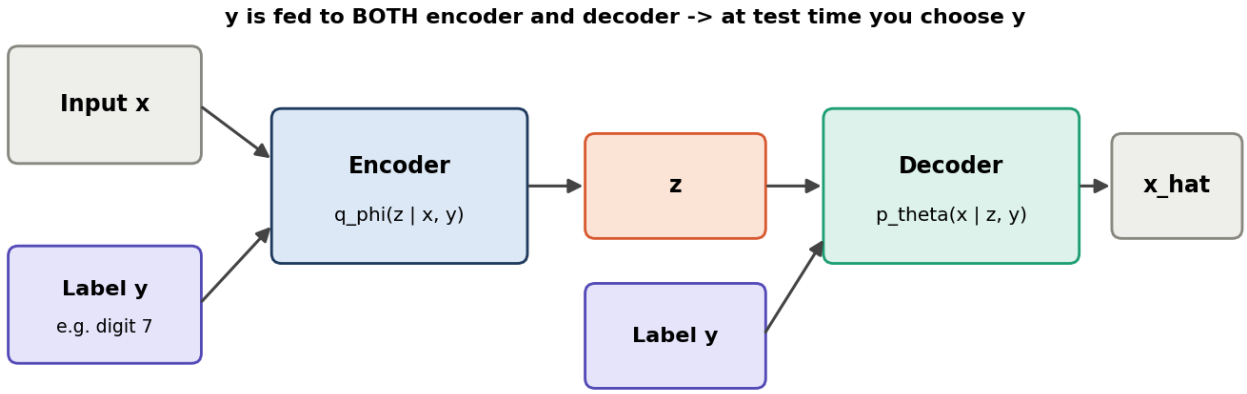


Figure 39. *A CVAE threads the label y through both networks.*

FORMULA — CVAE objective

$$\text{ELBO}(x \mid y) = \mathbb{E}_{q_\varphi(z|x,y)}[\log p_\theta(x \mid z, y)] - D_{KL}(q_\varphi(z \mid x, y) \,\|\, p(z))$$

What each symbol means

$q_\varphi(z \mid x, y)$	encoder that sees both the input and the label
$p_\theta(x \mid z, y)$	decoder that sees both the code and the label
$p(z)$	still N(0, 1) in the simplest version

In plain words: exactly the VAE loss, except that every network is also told the label.

WHY DOES THIS WORK?

Because the decoder is always told the label, the latent code z no longer needs to store 'which digit'. It is free to store only *style*: slant, thickness, size. At generation time you choose y to pick the class, and sample z to pick the style.

Generating a chosen class

```
z = torch.randn(1, latent_dim)                    # random style
y = F.one_hot(torch.tensor([7]), 10).float()       # ask for a 7
sample = model.decoder(torch.cat([z, y], dim=1))
```

ROBOTICS CONNECTION

In robot motion planning, a CVAE can be conditioned on the start and goal position (y) and trained on good trajectories (x). Sampling many z values then gives many different valid paths to the same goal, which a planner can check and choose from.

Chapter 17: β-VAE and Disentanglement

One extra number that trades sharpness for interpretability.

17.1 What disentanglement means

In an ideal latent space, each dimension would control one independent, interpretable factor: one dimension for rotation, one for stroke thickness, one for slant. In a plain VAE, dimensions are often **entangled**: moving one dimension changes several visual properties at once.

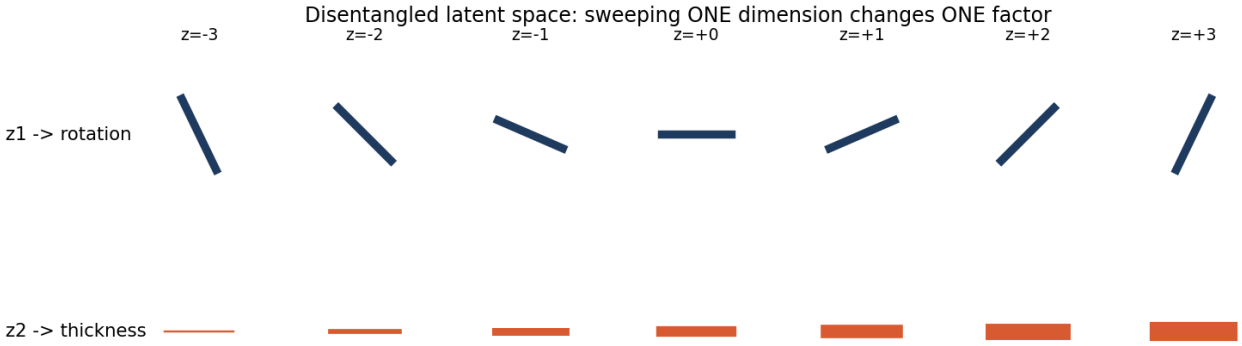


Figure 40. *Disentangled: sweeping z1 changes only rotation; sweeping z2 changes only thickness.*

17.2 The β-VAE loss

FORMULA — β-VAE

$$L_\beta(x) = \text{Reconstruction_Loss}(x, \hat{x}) + \beta \cdot D_{KL}(q_\varphi(z|x) \| p(z))$$

What each symbol means

β (beta)	a scalar weight on the KL term. β = 1 is the ordinary VAE
β > 1	stronger push towards the factorised prior: more disentanglement, blurrier outputs
β < 1	weaker push: sharper reconstructions, less regular space, less risk of collapse

In plain words: the same VAE loss, but with a dial that says how expensive it is to store information in z.

Why it is built this way: the prior N(0, I) treats each dimension as independent. Pushing q harder towards it makes the encoder avoid spreading one real-world factor over several correlated dimensions, nudging it towards one factor per dimension.

17.3 The trade-off

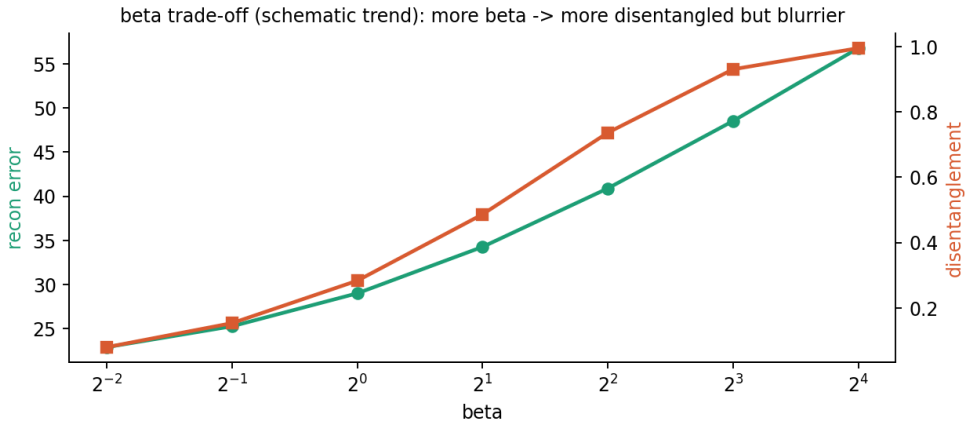


Figure 41. *Schematic trend: as β grows, disentanglement rises but so does reconstruction error.*

IMPORTANT - FOCUS HERE

β is a trade-off, not a free improvement. A very large β (say 50) gives blurry, generic reconstructions because the price of storing information is so high that the encoder keeps almost nothing. Choose β for your goal: interpretability or fidelity.

Chapter 18: VQ-VAE — Discrete Latent Codes

Replacing continuous clouds with a learned vocabulary.

18.1 Why discrete latents?

Some data is naturally discrete: language is made of tokens, and images often reuse a 'vocabulary' of textures and parts. The **Vector-Quantised VAE** replaces the continuous Gaussian latent space with a **codebook** of K learned vectors.

18.2 How it works

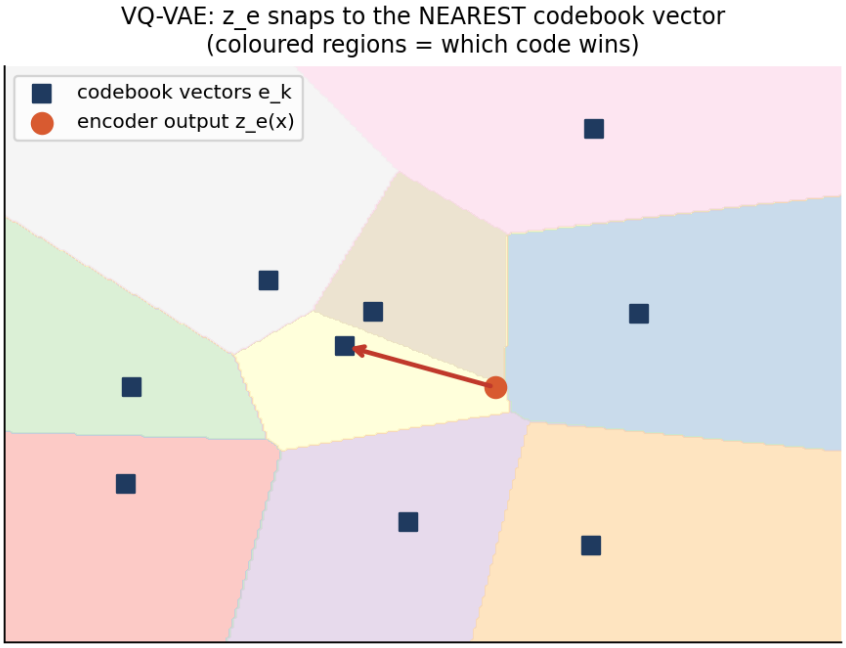


Figure 42. *The encoder output (orange) is replaced by the nearest codebook vector (navy square).*

FORMULA — Quantisation

$$k = \text{argmin}_i \, \| z_e(x) - e_i \| \; z_q(x) = e_k$$

What each symbol means

$z_e(x)$	continuous encoder output
e_i	the i-th codebook vector, i = 1 … K
argmin	the index of the closest codebook vector
$z_q(x)$	the quantised code the decoder receives

In plain words: snap the encoder's output to the nearest word in a learned dictionary.

FORMULA — VQ-VAE loss

$$L = \|x - \hat{x}\|^2 + \| \text{sg}[z_e(x)] - e_k \|^2 + \beta \cdot \| z_e(x) - \text{sg}[e_k] \|^2$$

What each symbol means

sg[·]	stop-gradient: treat this as a constant during backprop
2nd term (codebook loss)	moves the chosen codebook vector towards the encoder output
3rd term (commitment loss)	keeps the encoder output close to its chosen code, weight $\beta \approx 0.25$

In plain words: rebuild well, pull the dictionary words towards what the encoder produces, and make the encoder commit to its word.

The argmin is not differentiable, so VQ-VAE uses the **straight-through estimator**: on the backward pass, the gradient arriving at z_q is copied directly to z_e, as if quantisation had not happened. In code: `z_q = z_e + (z_q - z_e).detach()`.

WHY DOES THIS WORK?

There is no KL term in the standard VQ-VAE loss. With a uniform prior over K codes, the KL is a constant (log K), so it has no gradient and is dropped. Regularisation comes from the finite codebook itself.

KEY IDEA

Discrete codes are what made DALL-E (the original) and neural audio codecs practical: once data is a sequence of tokens from a small vocabulary, you can train a Transformer on those tokens. This ties the book back to where your journey started.

18.3 Codebook collapse

VQ-VAE's own failure mode is **codebook collapse**: only a few of the K vectors are ever chosen, and the rest are 'dead'. This wastes capacity and reduces diversity. Fixes: re-initialise dead codes to encoder outputs of random training examples, and update the codebook with an exponential moving average (EMA) instead of plain gradient descent.

Chapter 19: VAEs vs. GANs vs. Diffusion vs. Flows

Four ways to solve the same problem, and how they combine in modern systems.

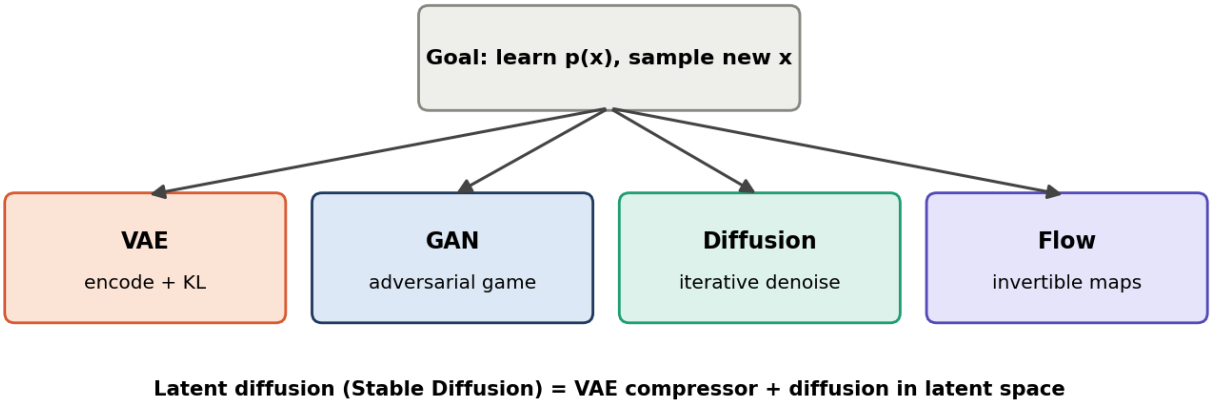


Figure 43. *Four families that all try to learn p(x) and sample from it.*

19.1 GANs

A GAN trains two networks against each other. The **generator** turns random noise into fake samples; the **discriminator** tries to tell real from fake. Each improves by competing with the other. GANs produce very sharp images, but training is unstable (mode collapse, vanishing gradients), they give no likelihood estimate, and they have no built-in encoder.

19.2 Diffusion models

A diffusion model learns to reverse a gradual noising process. It starts from pure noise and removes a little noise at each of many steps (often 20 to 1000) until a clean sample appears. Samples are high quality and diverse, and training is stable. The price is slow generation. Its loss is derived with the *same ELBO machinery* as Chapter 10, applied over many steps.

19.3 Normalising flows

A flow builds a generator from a chain of **invertible** transformations of a simple base distribution. Because each step can be undone exactly, the **exact** log p(x) can be computed with the change-of-variables formula, with no bound. The price is that every layer must be invertible, which limits architecture choices.

19.4 Comparison

Property	VAE	GAN	Diffusion	Flow
Training stability	High	Low–medium	High	High
Sample sharpness	Medium (can blur)	High	Very high	Medium–high
Diversity	High	Medium (mode collapse)	High	High
Built-in encoder	Yes	No	Approx. (inversion)	Yes (exact inverse)
Generation speed	Fast, one pass	Fast, one pass	Slow, many steps	Fast
Likelihood	Lower bound (ELBO)	None	Lower bound	Exact

19.5 The deep connection: latent diffusion

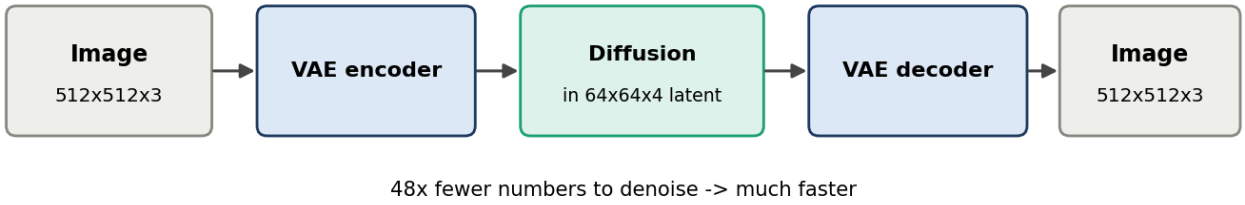


Figure 44. *Stable Diffusion: a VAE compresses images, diffusion runs in the small latent space, and the VAE decoder expands the result.*

KEY IDEA

Modern image generators are not 'VAE or diffusion' but VAE and diffusion. The VAE's encoder, decoder, latent space and reconstruction loss from Part III are reused inside state-of-the-art systems. A 512×512×3 image has 786,432 numbers; its 64×64×4 latent has 16,384, which is 48 times fewer to denoise.

PART V

Applications: robotics, autonomous systems, and latent-space exploration

Chapter 20: VAEs in Robotics, RL, and Autonomous Systems

Where the latent space becomes the bridge between perception and control.

20.1 World models

Raw sensor data (camera images, LiDAR point clouds) is very high-dimensional, so learning a policy or planning directly on it is expensive. **World models** (Ha and Schmidhuber, 2018) first compress each observation with a VAE into a small vector z, and then learn how z changes over time when the robot acts.

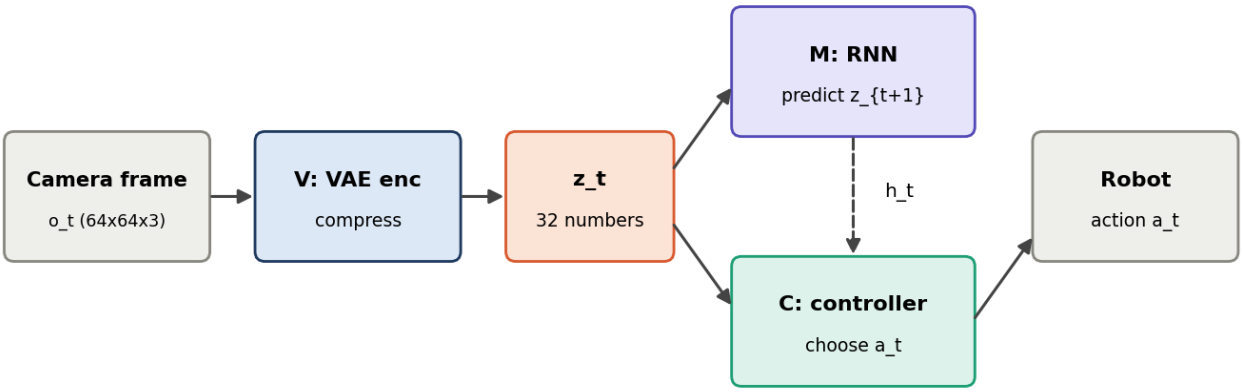


Figure 45. *The V–M–C world-model pipeline: Vision (VAE), Memory (RNN dynamics), Controller.*

- **V (vision):** a VAE encoder compresses each frame o_t into z_t.
- **M (memory / dynamics):** a recurrent network predicts the next code z_{t+1} from z_t, the action a_t and its hidden state h_t.
- **C (controller):** a small policy reads $[z_t, h_t]$ and chooses the action. Because its input is tiny, it is cheap to train, even inside the 'dream' of the learned model.

WHY DOES THIS WORK?
A VAE, not a plain autoencoder, is used because the dynamics model must predict and even sample future codes. That only works if the latent space is continuous and has no holes, which is exactly what the KL term guarantees.

20.2 Latent action spaces

For robots with many joints, such as a humanoid, learning a policy over the raw action space is hard. A VAE trained on short action sequences or trajectories gives a small, smooth **latent action space**. The policy outputs a low-dimensional latent action, and the decoder expands it into a coordinated whole-body command. This usually produces smoother, more physically plausible motion.

20.3 Anomaly detection and the sim-to-real gap

A trained VAE rebuilds familiar data well and unfamiliar data badly. So reconstruction error is an anomaly score. Set a threshold (for example the 99th percentile of error on validation data) and raise an alarm above it. This can flag sensor faults, unexpected obstacles, or real-world inputs that differ from the simulation the robot was trained in.

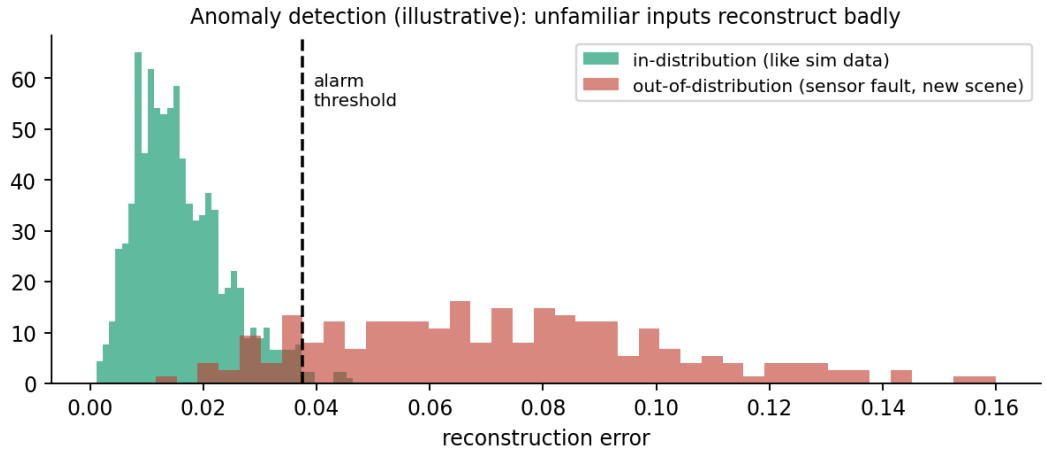


Figure 46. *Illustrative error distributions: in-distribution inputs have low error; unfamiliar inputs have high error.*

ROBOTICS CONNECTION

For your ROS 2 / Gazebo humanoid and TurtleBot projects: train a VAE on simulated camera frames, then run it on the real robot's frames. Frames with high reconstruction error show you exactly where your simulator is unrealistic, which tells you what to fix (textures, lighting, sensor noise) to close the sim-to-real gap. The same latent z can also be the compact state for an MPC or RL controller, and a VLA pipeline can use a VAE-style latent as the bridge between perception and the action head.

Chapter 21: Latent Space Interpolation and Visualisation

Visual proof that the latent space is continuous.

Pick two latent codes z_A and z_B (for example by encoding two real images and taking their means μ), then walk in a straight line between them and decode each point.

FORMULA — Linear interpolation

$$z(\alpha) = (1 - \alpha) \cdot z_A + \alpha \cdot z_B, \ \alpha \text{ from 0 to 1}$$

What each symbol means

α (alpha)	how far along the path: 0 = at A, 1 = at B, 0.5 = halfway

In plain words: mix the two codes in changing proportions and decode each mixture.

```
z_A = model.encode(image_A)[0]           # use the mean mu as a point estimate
z_B = model.encode(image_B)[0]
for alpha in torch.linspace(0, 1, steps=8):
    z = (1 - alpha) * z_A + alpha * z_B
    frame = model.decoder(z)                # smooth, meaningful morph
```

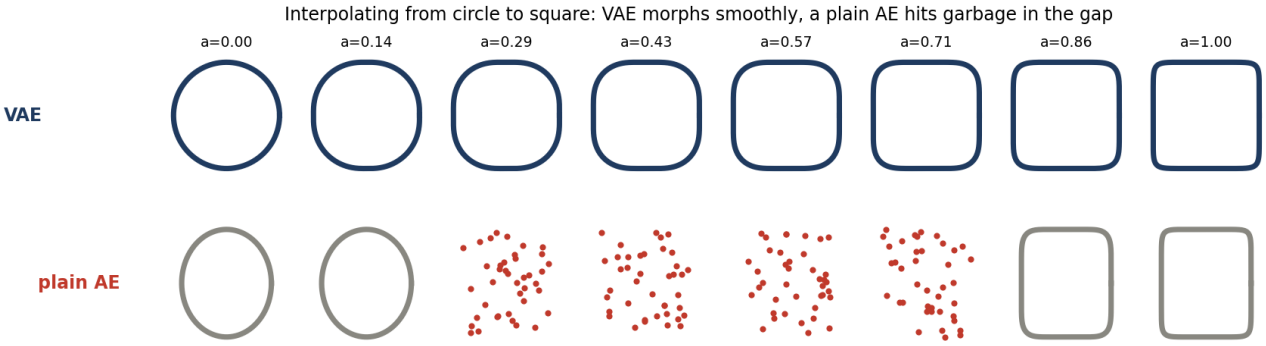


Figure 47. *A VAE morphs smoothly (top). A plain autoencoder passes through holes and produces garbage mid-way (bottom).*

IMPORTANT - FOCUS HERE

In high dimensions, most of the probability of N(0, I) lies near a shell of radius about √d, not at the centre. A straight line between two codes can pass through the low-radius middle, which is slightly atypical. **Spherical interpolation (slerp)**, which walks along the arc, often gives cleaner frames for large latent sizes.

Other ways to inspect the latent space: plot 2-D latent codes coloured by class (or use t-SNE/UMAP for more dimensions), and draw a grid of decodings by sweeping one dimension at a time while holding the others fixed (a 'latent traversal').

Chapter 22: Where to Go From Here

A practical study order.

1. **Implement a VAE from scratch** on MNIST or Fashion-MNIST using Chapter 12. Print the KL per batch, look at reconstructions, prior samples and interpolations.
2. **Implement β-VAE** and compare latent traversals at $\beta = 1$, 4 and 10.
3. **Study GANs:** implement a small DCGAN and compare its stability and sharpness with your VAE.
4. **Study diffusion models:** read the DDPM paper. Its loss uses the same ELBO machinery as Chapter 10, over many steps.
5. **Read World Models** (Ha and Schmidhuber, 2018) for the robotics connection in Chapter 20.
6. **Explore normalising flows** for exact likelihoods using invertible networks.

KEY IDEA

Every generative model you will meet answers the same question this book started with: how do we represent data compactly enough, and smoothly enough, to generate new and realistic examples? You now know one complete, rigorous answer.

PART VI

Advanced topics, evaluation, and practice

Chapter 23: Tighter Bounds — IWAE and Hierarchical VAEs

Closing the gap between the ELBO and log p(x).

23.1 The looseness of the plain ELBO

The gap between the ELBO and log p(x) is KL(q ‖ true posterior). If q is a simple diagonal Gaussian but the true posterior is complicated (for example, has two bumps), that gap can stay large however well we optimise.

23.2 Importance-Weighted Autoencoders (IWAE)

FORMULA — IWAE bound

$$L_K(x) = E[\ \log\ (\ (1/K) \cdot \Sigma_{k=1..K}\ p_\theta(x, z_k) / q_\varphi(z_k|x)\ )\ ],\ z_k \sim q_\varphi(z|x)$$

What each symbol means

K	number of samples drawn from the encoder per input
$p(x, z_k) / q(z_k|x)$	an 'importance weight': how much better this sample explains x than q expected
(1/K) Σ	average the weights before taking the log

In plain words: instead of one guess, take K guesses, average how well they explain x, and only then take the log.

Why it is built this way: the log of an average is larger than the average of logs (Jensen's inequality), so averaging first gives a tighter bound. With K = 1 this is exactly the ordinary ELBO; as K → ∞ it approaches log p(x).

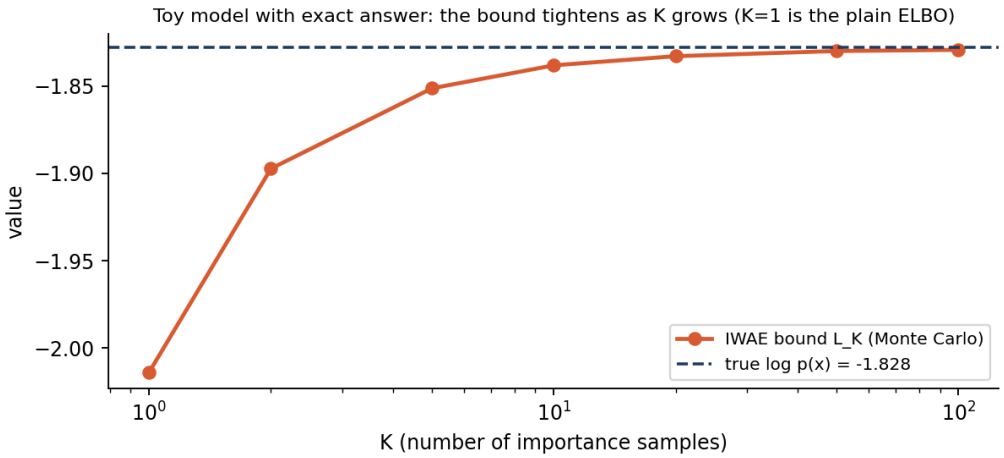


Figure 48. *A real computation on a toy model where log p(x) is known exactly (dashed line). With a deliberately imperfect encoder, the IWAE bound rises from the ELBO (K = 1) towards the true value.*

The cost is K times more encoder samples and decoder passes per step.

23.3 Hierarchical VAEs

FORMULA — Hierarchical generative model

$$p_\theta(x, z_1, \ldots, z_L) = p(z_L) \cdot p_\theta(z_{L-1} \mid z_L) \cdot \ldots \cdot p_\theta(z_1 \mid z_2) \cdot p_\theta(x \mid z_1)$$

In plain words: generate in stages: first a coarse top-level code, then progressively finer codes, then the data.

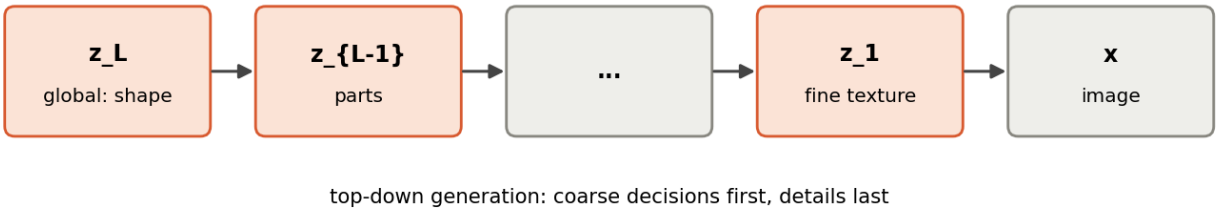


Figure 49. *Stacked latent layers: top layers decide global structure, lower layers add detail.*

Modern hierarchical VAEs such as NVAE and Very Deep VAE (VDVAE) use dozens of latent layers and produce much sharper samples than a single-layer VAE, while keeping the same ELBO training applied layer by layer.

Chapter 24: Evaluating Generative Models

No single number tells the whole story.

A classifier has accuracy. A generative model must be judged on a whole distribution. A model that memorises and replays training images would have perfect reconstruction but be useless as a generator, so you always need several checks.

24.1 Held-out reconstruction and ELBO

Reconstruction loss on validation data shows how well the model compresses and decompresses unseen data, but says nothing about new samples. The validation ELBO (or a tighter IWAE bound) estimates how well the model explains unseen data; a higher ELBO usually means a better model, though it does not always match visual quality.

24.2 Fréchet Inception Distance (FID)

FORMULA — FID

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right)$$

What each symbol means

μ_r, Σ_r	mean and covariance of Inception-network features of REAL images
μ_g, Σ_g	the same for GENERATED images
Tr	trace: the sum of the diagonal of a matrix

In plain words: describe the real and the generated images each by a Gaussian over deep features, then measure how far apart the two Gaussians are. The first term compares centres; the second compares spreads. Lower is better.

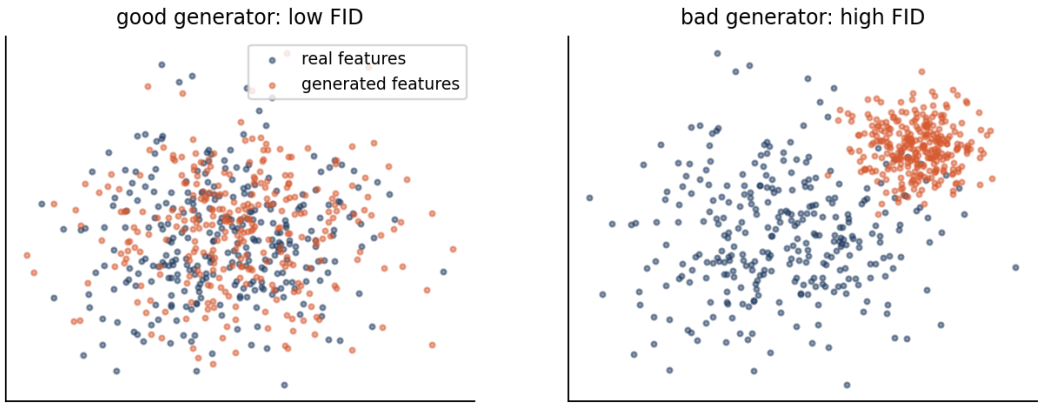


Figure 50. *Low FID: generated features overlap the real ones. High FID: they are shifted and too narrow (low diversity).*

24.3 Always look

Always inspect (1) reconstructions of held-out data, (2) fresh samples from the prior, and (3) an interpolation grid. Blur, all-samples-alike, or strange artefacts are usually obvious to the eye before any metric flags them.

Check	What it tells you
Held-out reconstruction loss	How well the model compresses and rebuilds real data
Held-out ELBO / IWAE bound	Estimated likelihood of unseen data
FID	Realism and diversity of generated samples
Interpolation (visual)	Whether the latent space is smooth and continuous
Prior samples (visual)	Whether sampling from N(0,1) gives plausible outputs
Average KL per dimension	Which dimensions are active; detects posterior collapse

Chapter 25: Exercises and Practice Problems

Try with pen and paper first, then check the solutions.

25.1 Conceptual questions

1. Explain in your own words why a plain autoencoder's latent space has gaps.
2. Why can't we compute the true posterior p(z|x)? Name the mathematical object that makes it intractable.
3. What does it mean that the ELBO is a lower bound on log p(x), and why is maximising it sensible?
4. Explain, without equations, what the reparameterisation trick does and why it is necessary.
5. A colleague sets $\beta = 50$ and gets blurry reconstructions. Explain why.
6. Why does VQ-VAE not need a KL term?
7. Why does the KL term rise during healthy training instead of falling?
8. In a CVAE, why does z end up storing 'style' rather than 'class'?

25.2 Computation practice

1. $\mu = 1.0$, $\log \sigma^2 = 0$. Compute the KL for this one dimension.
2. Same μ and $\log \sigma^2$, $\varepsilon = -0.5$. Compute z.
3. An encoder outputs $\mu = 0$ and $\log \sigma^2 = 0$ for every input. What does that tell you?
4. $\sigma^2 = 4$. What is $\log \sigma^2$, and what is σ?
5. $\mu = 0$, $\log \sigma^2 = -2$. Compute the KL. Which term is responsible?
6. Input $x = [1, 0]$, reconstruction $\hat{x} = [0.8, 0.3]$. Compute the sum-of-squares loss.
7. A conv layer has kernel 4, stride 2, padding 1, input width 64. What is the output width?

25.3 Design questions

1. You are building a VAE to compress LiDAR point clouds for a mobile robot's world model. What changes compared with the ConvVAE?
2. You need very sharp textures for a simulator and training time is unlimited. VAE, GAN or diffusion? Justify.
3. You want to generate only 'red cubes' on demand. Which variant solves this, and how?
4. Your VAE's KL is 0.001 for the whole run. List three things you would try.

25.4 Solutions

Conceptual

1. The loss only requires each training point to map to some code that decodes back to it. Nothing constrains the space between codes, so large regions are never used and the decoder never learns what to output there.
2. The evidence $p(x) = \int p(x|z)p(z)\,dz$, an integral over all z, has no closed form with a neural decoder, and the posterior divides by it.
3. $\log p(x) = \text{ELBO} + \text{KL}(q \| \text{posterior})$ and the KL is ≥ 0, so the ELBO is a floor. Raising it raises a guaranteed minimum for $\log p(x)$ and pulls q towards the true posterior.
4. It rewrites random sampling as a deterministic formula $z = \mu + \sigma\varepsilon$ of parameter-free noise ε, so the chain rule can pass gradients from the loss back to μ and σ.
5. With $\beta = 50$ storing information in z is extremely expensive, so the encoder keeps almost none; the decoder outputs averages, which look blurry.
6. Its latent is discrete with a uniform prior, so the KL is the constant $\log K$, which has zero gradient. The codebook and commitment losses regularise instead.
7. At the start the encoder outputs the prior for every input (KL ≈ 0, no information). As it learns to encode useful differences, it moves clouds away from the prior, which costs KL.
8. The decoder always receives the label, so z does not need to encode it; spending KL on the class would be wasted cost.

Computation

1. $\sigma^2 = 1$. KL $= \tfrac{1}{2}(1 + 1 - 0 - 1) =$ **0.5**.
2. $\sigma = 1$, so $z = 1.0 + 1 \times (-0.5) =$ **0.5**.
3. Posterior collapse: q equals the prior for every input, so z carries no information about x.
4. $\log 4 \approx$ **1.386**, and $\sigma =$ **2**.
5. $\sigma^2 = e^{-2} \approx 0.1353$. KL $= \tfrac{1}{2}(0 + 0.1353 + 2 - 1) =$ **0.568**. The $-\log \sigma^2$ term (+2) dominates: the cloud is too narrow.
6. $(0.2)^2 + (-0.3)^2 = 0.04 + 0.09 =$ **0.13**.
7. $\lfloor (64 + 2 - 4)/2 \rfloor + 1 = 31 + 1 =$ **32**.

Design (sample reasoning)

1. Point clouds are unordered and irregular, so 2-D convolutions are a poor fit. Voxelise and use 3-D convolutions, project to a range image and use 2-D convolutions, or use a PointNet-style encoder with a Chamfer-distance reconstruction loss. The ELBO and reparameterisation stay the same.
2. Diffusion: sharpest textures and stable training; slow sampling is acceptable here.
3. A Conditional VAE (Chapter 16): condition encoder and decoder on y = 'red cube', fix y and sample z to get varied red cubes.
4. KL annealing from 0, free bits (e.g. 0.25 nats per dimension), a weaker or less autoregressive decoder, and checking that the reconstruction and KL use the same reduction.

APPENDICES

Quick reference for revision

Appendix A: Frequently Asked Questions

Is z a single vector or a distribution?

Both, at different moments. $q_\varphi(z|x)$ is a distribution defined by μ and σ. A specific z is one vector sampled from it. When people say 'the latent representation of x' they usually mean μ.

Why is the prior N(0, 1)?

It is easy to sample from, has a closed-form KL against any Gaussian, and gives a known, bounded region to sample from. Other priors exist (mixtures, learned priors, VQ-VAE's codebook) for special cases.

Is a bigger latent dimension always better?

No. Too small cannot capture enough variation; too large leaves many unused or poorly regularised dimensions. It is a hyperparameter to tune; look at the KL per dimension to see how many are active.

Why are VAE samples blurrier than GAN samples?

Pixel-wise losses reward the average of plausible outputs when the model is uncertain, and an average of sharp images is blurry. A GAN's discriminator directly punishes blur.

Is the ELBO ever equal to log p(x)?

Yes, exactly when $q_\varphi(z|x)$ equals the true posterior, making the gap zero. In practice it gets close but never exact.

Should I sample z or use μ at test time?

Use μ for a deterministic encoding (interpolation, features for a controller). Sample z when you want variation, and sample from the prior for brand-new data.

Appendix B: Notation Reference

Symbol	Meaning
x	an observed data point (image, scan, trajectory)
$\hat{x}$	the decoder's reconstruction of x
z	a latent code (latent vector)
φ	ENCODER parameters (inference network)
θ	DECODER parameters (generative network)
$p(z)$	the prior over codes, usually $N(0, I)$
$p_\theta(x\|z)$	decoder / likelihood
$q_\varphi(z\|x)$	encoder / approximate posterior
$p_\theta(z\|x)$	the TRUE posterior (intractable)
μ, σ	mean and standard deviation from the encoder
$\log \sigma^2$	log-variance, the raw encoder output
ε	standard normal noise in the reparameterisation trick
$D_{KL}(\cdot\|\cdot)$	Kullback–Leibler divergence
ELBO	Evidence Lower BOund, the quantity maximised
β	weight on the KL term (β-VAE, annealing)
K	number of importance samples (IWAE) or codebook size (VQ-VAE)

Appendix C: Glossary

- **Autoencoder** — a network trained to reconstruct its own input through a narrow bottleneck.
- **Latent space** — the low-dimensional space where compressed codes live.
- **Bottleneck** — the smallest layer of an encoder–decoder network.
- **Generative model** — a model that learns to produce new samples resembling its training data.
- **Self-supervised learning** — training with targets made automatically from the data itself.
- **Prior** — the assumed distribution over codes before seeing data, e.g. N(0, 1).
- **Posterior** — the distribution over codes after seeing a particular data point.
- **Evidence** — p(x), the overall probability of a data point under the model.
- **Variational inference** — approximating an intractable distribution with the best member of a simpler family.
- **Amortised inference** — using one network to produce the approximate posterior for any input.
- **KL divergence** — a non-symmetric measure of how different two distributions are; always ≥ 0.
- **ELBO** — a computable lower bound on log p(x), maximised instead of the intractable likelihood.
- **Reparameterisation trick** — writing $z = \mu + \sigma\varepsilon$ so gradients can pass through sampling.
- **Posterior collapse** — failure where the decoder ignores z and the KL falls to zero.
- **Disentanglement** — each latent dimension controls one independent, interpretable factor.
- **Codebook** — VQ-VAE's finite set of learned vectors used to discretise the latent space.
- **World model** — a learned compact model of an environment's dynamics used for planning or control.

Appendix D: Full Formula Cheat Sheet

Every formula in the book on two pages. Revise from here.

Name	Formula	Remember it as
Encoder / decoder	$z = f_\varphi(x)$, $\hat{x} = g_\theta(z)$	compress / rebuild
MSE	$\Sigma_i (x_i - \hat{x}_i)^2$	square the errors, add them
BCE	$-\Sigma[x \log \hat{x} + (1-x) \log(1-\hat{x})]$	punishes confident wrong answers
Gaussian	$N(\mu, \sigma^2)$	μ = centre, σ = width
KL (general)	$E_q[\log q - \log p]$	average log-ratio under q
KL to N(0,1)	$\frac{1}{2}\,\Sigma(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$	zero only at μ=0, σ=1
Bayes' rule	$p(z\|x) = p(x\|z)\, p(z) / p(x)$	posterior = likelihood × prior / evidence
Evidence	$p(x) = \int p(x\|z)\, p(z)\, dz$	intractable
Encoder (VAE)	$q_\varphi(z\|x) = N(\mu_\varphi(x), \sigma_\varphi(x)^2)$	a cloud per input
Master identity	$\log p(x) = \text{ELBO} + KL(q \parallel p(z\|x))$	want = computable + gap
Lower bound	$\log p(x) \geq \text{ELBO}$	because KL ≥ 0
Practical ELBO	$E_q[\log p_\theta(x\|z)] - KL(q_\varphi(z\|x) \parallel p(z))$	rebuild well, stay near prior
VAE loss	Recon + KL	−ELBO
Reparameterisation	$z = \mu + \sigma\varepsilon$, $\varepsilon \sim N(0,1)$	$\partial z/\partial\mu = 1$, $\partial z/\partial\sigma = \varepsilon$
Std from log-var	$\sigma = \exp(\frac{1}{2} \log \sigma^2)$	always positive
KL in code	$-0.5 \cdot \Sigma(1 + \text{logvar} - \mu^2 - \exp(\text{logvar}))$	same formula, signs flipped
β-VAE	Recon + $\beta \cdot$KL	dial for disentanglement
Free bits	$\Sigma_j \max(\lambda, KL_j)$	free information budget
CVAE	$q(z\|x,y)$, $p(x\|z,y)$	give both networks the label
VQ	$z_q = e_k$, $k = \text{argmin} \|z_e - e_i\|$	snap to nearest code
Interpolation	$(1-\alpha)z_A + \alpha z_B$	walk from A to B
IWAE	$E[\log (1/K)\Sigma\, p(x,z_k)/q(z_k\|x)]$	average then log; K=1 is ELBO
FID	$\|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{\frac{1}{2}})$	distance between feature Gaussians

Appendix E: Debugging Checklist for VAE Code

Symptom first, then what to check.

Loss becomes NaN almost immediately

- Are you using σ = exp(0.5·logvar), and not treating the raw output as σ or σ²?
- Is logvar exploding? Clamp it, e.g. logvar.clamp(-10, 10), or initialise the last layer near zero.
- Does the loss match the output range (sigmoid + BCE for [0,1] data)?

Reconstructions are blurry and all samples look alike

- Print the average KL per batch. Near zero for the whole run means posterior collapse (Chapter 13).
- Try KL annealing or free bits.
- Check the decoder is not powerful enough to ignore z.

Samples from the prior look like garbage

- Is the KL actually added to the loss that you call .backward() on?
- Are you sampling from N(0, I) with the correct latent size?
- Was the model trained long enough for the KL to shape the space?

Training loss falls but generated samples do not improve

- Are you looking at prior samples, not reconstructions? They can differ a lot.
- Do both terms use the same reduction ('sum' and 'sum')? Mixing mean and sum silently re-weights the KL.

TRY IT YOURSELF
Three unit tests worth writing: (1) KL(μ=0, logvar=0) returns exactly 0; (2) for fixed ε, the gradient of z with respect to μ is 1; (3) the model's output shape equals the input shape.

Appendix F: Further Reading

- Kingma and Welling (2013). *Auto-Encoding Variational Bayes.* The original VAE paper.
- Rezende, Mohamed and Wierstra (2014). *Stochastic Backpropagation and Approximate Inference in Deep Generative Models.*
- Doersch (2016). *Tutorial on Variational Autoencoders.* Very readable companion.
- Kingma and Welling (2019). *An Introduction to Variational Autoencoders.* A longer modern overview.
- Higgins et al. (2017). *β-VAE.*
- Burda, Grosse and Salakhutdinov (2015). *Importance Weighted Autoencoders.*
- van den Oord, Vinyals and Kavukcuoglu (2017). *Neural Discrete Representation Learning* (VQ-VAE).
- Vahdat and Kautz (2020). *NVAE.* Child (2021). *Very Deep VAEs.*
- Goodfellow et al. (2014). *Generative Adversarial Networks.*
- Ho, Jain and Abbeel (2020). *Denoising Diffusion Probabilistic Models.*
- Rombach et al. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models.*
- Ha and Schmidhuber (2018). *World Models.*

Appendix G: The Whole Book in One Page

1. An **encoder** compresses x into a small code z; a **decoder** rebuilds $\hat{x}$ from z.
2. An **autoencoder** trains both to make $\hat{x}$ close to x. The bottleneck forces it to keep what matters.
3. Its latent space has **gaps**, because the loss never mentions the space between codes, so random codes decode to garbage.
4. A **VAE** encodes each input to a Gaussian cloud $N(\mu, \sigma^2)$ and pulls every cloud towards $N(0, 1)$, filling the gaps.
5. We want $\log p(x)$ but cannot compute it. We maximise the **ELBO** instead, which is always below it because $KL \geq 0$.
6. ELBO = reconstruction − $KL(q \| \text{prior})$. Loss = reconstruction loss + $\frac{1}{2}\Sigma(\mu^2 + \sigma^2 - \log \sigma^2 - 1)$.
7. The **reparameterisation trick** $z = \mu + \sigma\varepsilon$ lets gradients pass through sampling.
8. The two loss terms **fight**; the compromise is a smooth latent space. If the KL goes to zero, that is **posterior collapse**.
9. To generate: sample $z \sim N(0, 1)$ and run the decoder.
10. Variants: **CVAE** (control), **β-VAE** (disentangle), **VQ-VAE** (discrete tokens), **IWAE** and hierarchical VAEs (tighter bounds).
11. VAEs live on inside **latent diffusion**, **world models** and robot anomaly detection.

KEY IDEA
If you can explain every line above to a friend without notes, you understand VAEs better than most practitioners.